

University of Sheffield



University of
Sheffield

IntelliPlaylist

A Smart Assistant for Personalized Music Playlists using Answer Set Programming and Natural Language Processing

Fraser Green

Supervisor: Navid Talebanfard

*A report submitted in fulfilment of the requirements
for the degree of MComp in Computer Science*

in the

School of Computer Science

May 11th, 2026

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Fraser Green

Signature: Fraser Green

Date: 11/05/2026

Abstract

This project introduces IntelliPlaylist, a playlist assistant that utilises answer set programming techniques to recommend music in a dynamic and ‘intelligent’ manner. This novel approach has the potential to change the way that we create playlists, moving away from static, one-off recommendations and towards a dynamic human-like approach.

Existing playlist solutions can automatically suggest songs, but they typically apply fixed recommendation rules and require manual effort from the user to reshape or refine a playlist when their preferences change. The proposed solution has the ability to dynamically select and rearrange tracks in order to flexibly accommodate updated listening goals and constraints without the need for detailed manual intervention.

The methods used to introduce flexibility into the playlist generation process are presented, alongside an attempt to encode a notion of human musical intuition through constraints on mood, flow and diversity. The impact of the underlying track dataset configurations and the user’s constraints on the solution’s performance is also analysed, with the goal of producing high-quality playlists within a few seconds for typical usage scenarios.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Aims	1
1.3	Objectives	2
2	Literature Survey	3
2.1	Introduction	3
2.1.1	Music Recommendation Systems	3
2.1.2	Types of Recommendation Systems	3
2.1.3	Limitations of Existing Music Recommendation Systems	4
2.2	The Dataset	4
2.3	Representing Music and Context for Recommendations	5
2.3.1	Audio Feature Representation	5
2.3.2	Emotion, Mood, and “Vibe” Representation	5
2.3.3	Context-aware and Activity-based Music Recommendation	6
2.4	Answer Set Programming	6
2.4.1	Logic Programming	6
2.4.2	Answer Set Solvers	7
2.4.3	Constraints, Choices, and Optimisation	7
2.4.4	Suitability in This Context	8
3	Requirements and Analysis	9
3.1	Project Requirements	9
3.1.1	Functional	9
3.1.2	Non-functional	10
3.2	Answer Set Programming Solver	10
3.2.1	Encoding Tracks and Playlists as an Answer Set Program	10
3.3	Natural Language Interpretation	10
3.4	Evaluation Technique	11
3.5	Ethical Considerations	11

4	Design	12
4.1	Design Overview	12
4.2	Dataset and Track Representation	13
4.2.1	Dataset Preparation	13
4.2.2	Feature Selection	14
4.2.3	Track Representation	14
4.3	Candidate Retrieval and Filtering	14
4.4	Answer Set Programming Solver	15
4.4.1	Converting tracks to ASP facts	16
4.4.2	ASP Rules and Solver	17
4.4.3	Constraint Types and Optimisation	18
4.4.4	Post-processing and Practical Handling	21
4.5	Natural-Language Request Interpretation	21
4.5.1	Using LLMs to Convert Natural Language to a Structured Playlist Request	22
4.5.2	Parser Fallback and Pipeline Integration	23
4.6	Web Application and User Interface	23
4.6.1	Application Structure	23
4.6.2	Front-End	24
4.6.3	Back-End	24
5	Implementation and testing	26
5.1	Dataset and Candidate Retrieval	26
5.1.1	Dataset Import	26
5.1.2	Candidate Retrieval	27
5.2	Natural-Language Parser	27
5.2.1	Using LLMs to Convert Natural Language to ASP	28
5.2.2	Using LLMs to Convert Natural Language to a Structured Playlist Request	31
5.3	ASP Solver and Web Application	32
5.3.1	Clingo Integration	32
5.3.2	ASP Encoding Considerations	33
5.3.3	Web application	33
5.4	Functional Testing	36
5.4.1	Natural-language parser	36
5.4.2	Candidate Retrieval	37
5.4.3	ASP solver	38
5.4.4	Post-processing	39
5.4.5	Summary	39

<i>CONTENTS</i>	5
6 Results and discussion	40
6.1 Comparison Study and Findings	40
6.2 Requirements Analysis	41
6.2.1 Functional Requirements	41
6.2.2 Non-Functional Requirements	42
6.3 Challenges and Further Work	42
7 Conclusion	43
A Evaluation Data	45
A.1 Per-playlist Ratings	45
A.2 Defects Observed During Evaluation	47
A.2.1 Duplicate Track Examples	47
A.2.2 Empty Candidate Pool	47

List of Figures

4.1	Overview of IntelliPlaylist Pipeline.	13
4.2	Diagram of the IntelliPlaylist application structure.	24
5.1	Example of prompt given to LLM to generate ASP from playlist description .	28
5.2	Program generated by LLM following prompt in Figure 5.1	29
5.3	One-shot learning prompt given to LLM to generate ASP from playlist description	30
5.4	Program generated by LLM following prompt in Figure 5.3 when queried . .	30
5.5	Extract from the prompt given to the LLM to produce a structured playlist request.	31
5.6	Structured playlist request produced from the LLM prompt in Figure 5.5. . .	32
5.7	Initial User Interface.	34
5.8	Request modal where users describe their desired playlist in natural language.	35
5.9	Generated Playlist View.	35
5.10	Structured playlist request from the description used in Figure 5.9.	36

List of Tables

3.1	Functional Requirements of the Project.	9
3.2	Non-functional Requirements of the Project.	10
5.1	Functional tests for the natural-language parser.	37
5.2	Functional tests for candidate retrieval.	37
5.3	Functional tests for the ASP solver.	38
5.4	Functional tests for post-processing.	39
6.1	Mean ratings across the ten requests for each measure.	40
6.2	Success of Functional Requirements of the project.	41
6.3	Success of Non-Functional Requirements of the project.	42

Chapter 1

Introduction

1.1 Motivation

Putting together a good playlist often feels like a surprisingly challenging experience. Consider the following situation: one wants a 1-hour playlist for studying, with calm, mostly instrumental tracks that do not change too much in energy. With existing automated solutions, one might start with a pre-made Spotify “study” playlist or an auto-generated mix based on a tracks they like. In reality, they end up skipping tracks that feel too energetic, removing songs with distracting vocals, and reordering tracks for a smoother flow. Were a human assistant to create the same playlist, they could naturally apply a set of informal rules. These could include putting similar songs in succession, spacing out tracks by the same artist, and ensuring energy level remains roughly consistent over the hour. The goal is to explore whether these intuitive rules can be generalised into explicit constraints and optimisation criteria, and to develop tools that solve them with human-like behaviour.

Current streaming platforms such as Spotify and Apple Music have transformed how we discover and listen to music, with their algorithms offering powerful recommendation features. However, they tend to operate as “black boxes”: users have little visibility into how a recommended playlist is generated, and then must perform the “Tetris” themselves by liking, skipping, or editing tracks. Furthermore, such recommendation systems are limited in their ability to process flexible, higher-level constraints. If a user’s preference were to change (e.g. from relaxed background music to an intense gym mix), they would be forced to start again, rather than the system intelligently reshaping the playlist around these new goals.

1.2 Aims

This work introduces **IntelliPlaylist**, a music assistant that uses answer set programming to create playlists in a dynamic and ‘intelligent’ manner. The system aims to simplify playlist creation by allowing users to generate playlists that align with their current mood, activity, or preferences. IntelliPlaylist converts natural language requests into music recommendations by reasoning about suitable song characteristics (tempo, energy, valence, etc.), a capability that

is not commonly supported in existing solutions. The methods used to introduce flexibility into the creation process are presented, alongside an attempt to encode a notion of human intuition. Later, the impact of different playlist constraints and dataset configurations on the efficiency of the solution will be analysed, with an evaluation of the generated playlists from a usability perspective.

1.3 Objectives

By the end of the project, the aim is to achieve the following set of high-level goals:

1. To define a constraint structure that allows for playlists to be encoded as a concise set of requirements.
2. To explore methods of introducing flexibility or uncertainty into the playlist generation process.
3. To utilise constraint-solving methods to efficiently and accurately construct playlists, with the ability to dynamically update selections while maintaining stability.
4. To design an algorithm that translates natural language playlist descriptions into the formal constraint language described in (1).

Chapter 2

Literature Survey

2.1 Introduction

2.1.1 Music Recommendation Systems

Recommender systems aim to predict information a user is likely to find interesting, changing their decision making process [22]. They have become a standard in many online platforms, helping users cope with information overload and reducing the effort needed to discover relevant content [6, 22].

Recommendation algorithms are particularly important in music, where streaming services give listeners access to a huge pool of data. For instance, Spotify is the most popular music streaming service and allows users to enjoy over 100 million tracks [25]. Users typically interact with only a tiny fraction of the Spotify catalogue and rely on the platform to recommend music. These systems use a hybrid approach to recommendation, applying more than one method to recommend songs and artists to a user [1].

2.1.2 Types of Recommendation Systems

The most commonly used filtering approaches used in recommendation systems are collaborative and content-based filtering.

Collaborative filtering evaluates information based on the user's previous behaviour and their historical data [6]. Suppose two users have a very similar listening history. If one user were to listen to a song that a similar user has never heard, then, following the collaborative filtering paradigm, the system will recommend this song to that user [4, 11].

Content-based filtering provides recommendations based on unseen items, utilizing various feature values and keywords [6]. In music, content-based features can be split into two sections. The first is high-level which consist of attributes such as artists, genre and tags (metadata). The second is low level which consists of attributes such as timbre, pitch, or tempo. These lower level features are extracted from the audio core using various preprocessing algorithms [15]. The system will recommend tracks based on the similarity to the content within the music that a user already listens to. This helps users discover new and nuanced tracks.

Other than collaborative and content-based approaches to recommendation, there are two further paradigms that could be relevant within the music domain. We could incorporate information about the user’s current situation, such as the time of day, device, location, or their activity. This is the context-aware approach, and it allows recommendations to align with the current moment, rather than just the long term taste [17]. For example, the system may target calmer tracks when a student is studying late at night but more aggressive tracks when a user is exercising in the morning. A constraint-based filtering approach uses rules, preferences and constraints over features to find a user their recommendations, rather than just relying primarily on historical data [7]. In music, this could be the tempo range, playlist length, or proportion of speechiness to instrumentalness in tracks. This is useful in situations where the user must satisfy various conditions, which is a primary focus of the approach in this project.

2.1.3 Limitations of Existing Music Recommendation Systems

Although current music recommendation systems from companies such as Spotify and Apple Music are highly effective in allowing users to break down and navigate large catalogues, they also have several limitations relevant to this project. A platform like Spotify combines collaborative filtering with content-based filtering, forming various different recommended playlists [5]. Although they produce incredibly accurate results and engaging recommendations for users, the behaviour of the system is largely opaque. A study of user folk theories about Spotify’s recommendations reported that “Participants believe Spotify is a complete black box” [23]. They say that the system hides things, and it’s difficult to steer the algorithm in a certain direction to shape future recommendations.

A second limitation behind these commercial systems is the difficulties of handling flexible, higher-level constraints and changing preferences. These traditional algorithms prioritise historical data like logs and ratings, giving a user limited control over results when their current situation or preferences changes [14]. This issue could easily be solved by giving users more control to tailor recommendations to their current moods and influences adaptively [16].

2.2 The Dataset

Large catalogues of music in the form of publicly available datasets are considerably more difficult to obtain than in other domains (such as image and text), so a number of datasets have become a standard for feature extraction and recommendation systems. One example is a Kaggle dataset containing audio features for over 1.2 million songs, based on the Spotify web API [8]. This dataset is particularly attractive due to its size, making it well suited for studying content based music recommendation over a wide range of artists. However, it lacks a popularity attribute, limiting its usefulness when distinguishing between familiar songs and more obscure songs. This project instead uses the Spotify Tracks Dataset [20], which includes a popularity attribute and is described in detail in Section 4.2.1.

2.3 Representing Music and Context for Recommendations

2.3.1 Audio Feature Representation

Music information retrieval (MIR) extracts numerical audio features for tasks such as classification and recommendation. These features are often split into low-level and high-level descriptors. Low-level audio features describe the spectrum and temporal change of the digital audio signal [28, 3]. However, many modern systems use higher-level audio descriptors. These descriptors take lower level features and make them more interpretable. Services such as the Spotify API provide standard attributes for tracks including acousticness, danceability, duration, energy, instrumentalness, key, liveness, loudness, mode, speechiness, tempo, time signature, type and valence [26]. These attributes are summaries of high-level acoustic patterns in terms of, for example, how suitable a track is for dancing (danceability), how intense it is (energy), how positive it feels (valence) etc.

High-level audio-feature-based representations are now used frequently in content-based and content-driven music recommendation for playlist creation. They give a more broken down description of each track that aligns with listener concepts such as mood, style and listening context. Building on this idea, Bogdanov et al, use a semantic descriptor space where every track is described using tags such as genre, instrumentation, rhythm and tempo, inferred from the lower-level audio features. Recommendations are then performed in this space, and Bogdanov shows that these higher-level descriptors result in a better recommendation performance than using raw low-level timbral features alone [2].

2.3.2 Emotion, Mood, and “Vibe” Representation

Emotions in music are often modelled to be either categorical or dimensional. In a categorical model, songs are assigned to a set of discrete labels such as happy, sad, angry, or chill, and sometimes organised into affective “rings” or taxonomies [29]. In dimensional approaches, emotions are described as points in a continuous space, rather than as separate categories. The most common example is the valence-arousal plane, introduced by Russell’s circumplex model of affect [24]. This organises affect along two axes: valence, which ranges from unpleasant to pleasant, and arousal, which ranges from low to high activation. Another influential model is Thayer’s theory of energetic and tense arousal [27].

In music emotion recognition, these categorical and dimensional approaches are often tied directly to the audio feature space. A listener’s valence and arousal level is related to a combination of tempo, loudness, mode and harmony characteristics [30]. For example, slower and softer music tends to be associated with lower arousal and more negative valence levels, while faster and louder music is often perceived as highly arousing and, in many contexts, more positive. High-level features such as energy, valence, tempo, loudness, danceability, acousticness and instrumentalness can therefore be treated as musically and psychologically meaningful dimensions that correlate with perceived mood and “vibe”. In practice, playlist labels such as chill, study, workout or party can be seen as informal names for regions in this feature space, where particular combinations of valence, arousal and timbral-rhythmic

characteristics tend to co-occur.

2.3.3 Context-aware and Activity-based Music Recommendation

Context-aware music recommender systems take into account information about the user’s current situation, such as their activity, time of day, location, or social setting [17]. According to a study on the psychology of music in everyday life, a listener’s momentary needs are often shaped by their context. This could be their current activity, emotional state, the weather, or their environment [19]. For example, someone who is working out is likely to prefer louder, higher-energy music.

There are many ways to represent and exploit context from a user. One approach uses tags and signals such as playlist titles or mood/activity labels like studying, focussing or chilling [21], learning which tracks usually appear together under each theme. Another infers context from sensor data such as heart rate or motion [18], but this requires device instrumentation and does not expose the underlying feature-level constraints to the user.

However, these approaches learn implicitly from high level descriptions such as “I’m studying”, and the mapping to musical properties (such as tempo range and energy level) is not disclosed to the user. As a result, users cannot understand and adjust the feature-level constraints influencing their recommendations. For instance, they cannot specify requirements such as “Slightly faster, but keep the energy low and mainly instrumental”.

This motivates a constraint-based approach in which mood and context labels are mapped to explicit audio-feature ranges, as realised in IntelliPlaylist using Answer Set Programming.

2.4 Answer Set Programming

Answer Set Programming (ASP) is a declarative programming paradigm that expands on logical programming and non-monotonic reasoning, based on the concept of stable models [13]. Instead of encoding an algorithm, the ASP program encodes the problem itself. This is then passed to an answer set solver to compute a solution to satisfy a set of logical rules. ASP has been widely used in a range of research on artificial intelligence and for solving complex search and optimisation problems.

In the context of the project, ASP is used to describe playlist generation as a constraint satisfaction and optimisation problem over tracks and their audio features. This section will introduce the core concepts of ASP, and how it can be used in this way.

2.4.1 Logic Programming

Logic programming provides the foundation for ASP. Only the basic concept is summarised here; more detailed explanations can be found in Lifschitz (2019). Traditional logical programming, used in languages such as prolog, consist of a set of facts and rules. A fact is a basic statement

that is taken to be true. For example:

$$\textit{Country}(\textit{france}), \textit{country}(\textit{uk}) \quad (2.1)$$

$$\textit{country}(\textit{france}; \textit{uk}) \quad (2.2)$$

The two facts in (2.1) show that France and the UK are definitely countries. The short hand of the two facts can be seen in (2.2), which is known as the pooled version of (2.1). Rules contain a head and a body, separated by :- that reads as ‘if’. This can be seen in the following rule:

$$\textit{large}(C) : \textit{-size}(C, S1), \textit{size}(\textit{uk}, S1), S1 > S2 \quad (2.3)$$

$\textit{large}(C)$ is the head of the rule, and $\textit{size}(C, S1)$ is the body. This can be read as “A country C is large if the population of C is S1, the population of the UK is S2, and $S1 \not\leq S2$ ”. Commas in the body of a rule are equivalent to conjunctions used in logical formulas.

In practice, users can pose queries (e.g. $\textit{large}(C)?$), and the system will search the substitute to make the query true. Because this evaluation order is tied to the interpreter, logic programs are generally not fully declarative. However, ASP becomes fully declarative when stable model semantics are adopted. This means that a solution can be derived directly from the specification encoded in the ASP program, without the need for individual user queries. The software that performs this task is known as an answer set solver.

2.4.2 Answer Set Solvers

State of the art solvers such as Clingo combine the technologies of Gringo, a grounder which instantiates rules with concrete constants, and clasp, a solver based on conflict-driven search [9]. This is the technology that is used in most ASP technologies, and is therefore ideal going forward for the solution.

2.4.3 Constraints, Choices, and Optimisation

An answer set program also consists of constraints, choice rules, and optimization statements in addition to the key components of a logic program as described in Section 2.4.1. These are used for added influence over what is classed as a stable model.

In ASP, a constraint is a rule with an empty head, written for example as:

$$\text{:- tempo}(T, Bpm), Bpm > 140, \textit{context}(\textit{study}) \quad (2.4)$$

This constraint means that, under a study context, no stable model is allowed in which there exists a track T with tempo above 140BPM. If a constraint is violated, such as the tempo going over 140 BPM, any stable model is eliminated from the solution.

A choice rule specifies which atoms may be included in the stable model. Integers before

or after the braces bound how many of those atoms must hold. For example:

$$10(in_playlist(T) : candidate_{track}(T))20 \quad (2.5)$$

The choice rule (2.5) requires that between 10 and 20 out of the candidate tracks must hold in the stable model

Finally, an optimisation statement allows us to define the quality of a solution. It does this by assigning weights to express preferences and using an expression to combine these values. We can express these preferences through directives like minimize or maximise statements which aim to assign values to minimise or maximise the sum of weights respectively. Optimisation statements can also be linked with priorities, so when a program contains several optimisations, it can satisfy statements with higher priority first. For example:

$$maximize((V, T : in_playlist(T), valence(T, V)) \quad (2.6)$$

Optimisation statement (2.6) uses a shorthand form where each atom $valence(T, V)$ contributes to a weight of v to the object, and no explicit priority is given, so a default priority of 1 is used. This will result in the sum of the valence values of the selected tracks being as large as possible.

2.4.4 Suitability in This Context

Based on the approaches considered so far, an Answer Set Programming (ASP) approach appears to be well suited to the playlist generation problem. ASP is powerful and efficient for combinatorial search problems, which is the setting in which we need to select a subset of tracks that jointly satisfy a collection of constraints on audio features, artists, and playlist properties. It is also very well documented, which will become a major advantage when it comes to the development and experimentation process. The main limitation, however, is that ASP has no built-in representation of time or uncertainty. Nevertheless, both can be handled fairly easily by creating simple encodings to represent time as integers, and modelling preferences as soft constraints or optimisation criteria.

Chapter 3

Requirements and Analysis

3.1 Project Requirements

This chapter outlines the core requirements of the project, which are summarised in tables 3.1 and 3.2 using the MoSCoW prioritisation method.

3.1.1 Functional

Requirement	Priority
Implementation of natural language input interface that allows users to describe their desired playlist	Must Have
Implementation of user control over playlist length	Should Have
Implementation of a component that maps user descriptions to an internal emotion profile and corresponding audio features	Must Have
Implementation of a playlist generation feature that uses an Answer Set Programming Solver and the dataset of tracks to select tracks relevant to the computed feature ranges	Must Have
Display the generated playlist in a clear format	Must Have
Avoid generating playlists with duplicate tracks	Must Have
Implement the ability for users to specify multiple moods within one request	Should Have
Display multiple playlists for the same description to allow for comparison	Could Have

Table 3.1: Functional Requirements of the Project.

3.1.2 Non-functional

Requirement
The web app should be simple and friendly
Playlists should be generated within a short, interactive time
The web app should be responsive to a variety of screen sizes
The web app should be available across all major browsers

Table 3.2: Non-functional Requirements of the Project.

3.2 Answer Set Programming Solver

The core technology of this project is an Answer Set Programming (ASP) model to encode a track’s meta data and define user requirements into logic programming constructs. This is then passed to the Clingo answer set solver (introduced in Section 2.4.2) to evaluate preferences and return the best possible playlist.

This approach allows the problem of playlist creation to be expressed declaratively. Rather than writing an algorithm that constructs the playlist gradually, the system states what a valid playlist must have. The solver searches the catalogue of possible tracks, and returns those that meet the specifications.

3.2.1 Encoding Tracks and Playlists as an Answer Set Program

In the logic program, a track’s encoding is represented by facts that record its id and audio features (such as tempo, energy, valence, danceability, acousticness). A resulting playlist indicates which tracks have been selected by a set of atoms.

Initially, the system maps the inputted natural language description to an emotion profile and a corresponding audio feature range. These weighted ranges, and the desired playlist length, can then be injected into the ASP program as constraints. In practice, this means that the solver must select N tracks, ensure that every selected track has audio features within the ranges derived from the request, and avoid selecting the same track more than once. These three requirements are kept minimal at this stage, as the complexity of the problem lies in the soft constraints and optimisation discussed in Section 4.4.3 rather than in the hard selection logic itself.

3.3 Natural Language Interpretation

A key aim of IntelliPlaylist is to allow the user to describe the kind of playlist they would like in natural language, rather than specifying the numerical audio feature, or writing ASP code. Natural language interpretation acts as a bridge between the user’s description, and

the constraints required by the answer set programming solver. The process and schema are described in detail in Section 4.5.

3.4 Evaluation Technique

Two forms of evaluation will be used to assess IntelliPlaylist. The first is functional testing of the main pipeline components. The second is an evaluation of the generated playlists against a baseline. Functional testing will use a category-partition approach. This means that a small set of representative inputs will be chosen for each part of the system, covering both typical and edge-case behaviour. Playlist quality will be evaluated by comparing Intelliplaylist’s output against a random baseline that selects tracks uniformly from the dataset. Because recommendations are subjective, there is no single ‘correct’ playlist. For this reason, standard classification metrics such as recall or F1-score are not suitable. Instead, the following three methods will be used:

- **Precision:** how many of the recommended tracks are suitable for the user’s request;
- **Novelty:** how many of the recommended tracks are unfamiliar to the person rating the playlist. This shows whether the system can suggest less obvious songs;
- **Subjective quality:** an overall rating of how well each playlist matches the requested style. This will be on a five-point Likert scale;

Each of these metrics will be computed across a set of playlists created from a diverse set of natural-language requests, with the same requests used for both the Intelliplaylist and baseline conditions. The aggregate scores between the two will then be compared to assess whether IntelliPlaylist performs better than random selection. This is a smaller scale evaluation than a full user study. A formal study with external participants was outside the scope of the project. Therefore, the results should be treated as indicative rather than as statistically conclusive evidence.

3.5 Ethical Considerations

The main issue related to the project is data privacy particularly during the testing and evaluation of the application. Users will be entering natural language descriptions of the kind of music they want based on various situations - this means that the system may have access to personal details such as activities, moods, or even their whereabouts if they include their location (e.g. “walking home in Sheffield late at night”). Although most of the time, this information is relatively low risk and will not necessarily be visible to others, it is still very important that it is stored securely to ensure data privacy and security.

Chapter 4

Design

4.1 Design Overview

This section describes the design approach taken in the implementation of IntelliPlaylist. The design is based on the main aim of allowing users to create playlists through natural-language requests, which are then translated into a set of more formal constraints that the system can reason over. The design is centred around a constraint-based generation process using Answer Set Programming (ASP), supported by database filtering, natural-language processing, and a user interface. The system follows a hybrid pipeline consisting of five main components:

- track representation and database preparation (Section 4.2);
- candidate retrieval and database filtering (Section 4.3);
- constraint-based playlist selection using ASP (Section 4.4);
- natural-language request interpretation (Section 4.5);
- user interface and playlist presentation (Section 4.6);

Figure 4.1 shows the overall flow of the system. A user enters a natural-language request through the interface, which the natural-language parser converts into a structured JSON request, with a rule-based parser as a fallback if the LLM is unavailable. The structured request guides an SQLite query that retrieves a set of candidate tracks from the database. These are then encoded as facts for the Clingo answer set solver, which selects a final playlist that satisfies the hard constraints and optimises the soft constraints expressed in the request. The tracks are then ordered and constraints are relaxed if no valid playlist can be found. Finally the playlist is returned to the interface for display.

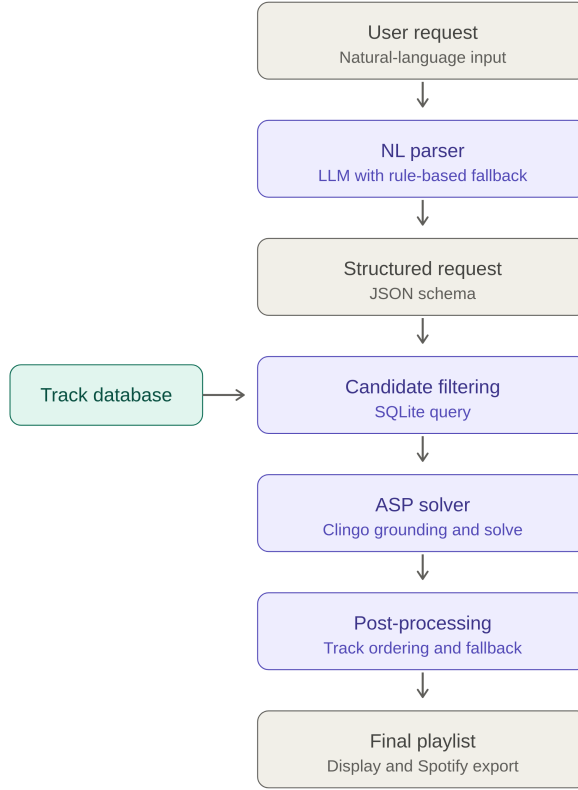


Figure 4.1: Overview of IntelliPlaylist Pipeline.

4.2 Dataset and Track Representation

4.2.1 Dataset Preparation

Before logical reasoning can begin, the raw Spotify track data must be prepared into a form that the pipeline can query. The dataset that IntelliPlaylist will use is the Spotify Tracks Dataset, published by maharshipandya in 2022 [20]. It contains 114,000 tracks including 125 genres, containing metadata and audio features obtained from the Spotify Web API.

In order to import the dataset into the database, the source CSV is processed offline and cleaned before insertion. Column names are mapped onto a consistent schema, any missing values are filled, and any rows without a valid ID or name are removed. This condensed dataset is then inserted into an SQLite database.

SQLite was chosen because it is lightweight, integrates directly into Python without a separate server, and suits the repeated numerical filtering needed to match tracks to feature ranges. Indexes are created over the name, album, artists, genre, popularity, and year columns to support the candidate-retrieval queries described in Section 4.3. Since Spotify audio features are already in numeric form, minimal additional preparation is needed before they are stored

or later translated into ASP facts.

4.2.2 Feature Selection

The system uses a subset of audio features to represent each track. The selected features are tempo, energy, valence, danceability, acousticness, instrumentality, liveness, and speechiness. These are supported by additional metadata such as track duration, popularity, year, genre and explicit status. Together, these attributes provide a compact and expressive numerical description of the musical properties that are most relevant to a playlist’s mood and activity, as well as the constraints expressed in the user request.

The core of the selected feature set is formed by tempo, energy, and valence. Tempo captures the pace of the track, energy reflects its recognised intensity, and valence provides an indication of its emotional tone. This makes these features prevalent in modelling broad requests such as calm, sad, or energetic, with the remaining features adding further detail to this representation: danceability helps to distinguish more active tracks; acousticness and instrumentality help identify softer or less-vocal tracks; liveness and speechiness refine tracks with stronger live-performance or spoken-word characteristics.

4.2.3 Track Representation

Each track is represented in a structured form that can be used consistently throughout the pipeline. This representation includes a unique identifier for the track, basic metadata such as track name, artist, album, genre, year, popularity, explicit status and duration, and the selected audio-feature values.

The unique identifier allows tracks to be referred to unambiguously across the system. Although metadata such as track names and artists are useful for display, they are not always sufficient enough for back-end referencing, as different song titles may be similar or identical.

Within the database, the track representation is used for candidate retrieval and filtering. The selected numerical audio features allow tracks to be compared against feature ranges, while the metadata supports filtering. This means that before playlist selection, the system can reduce the full dataset to a smaller candidate set.

When candidate tracks are passed into the ASP stage, they are converted into a fact-based representation that can successfully be reasoned over by the solver. However, the feature values must first be converted into integers to support ASP compatibility (Section 4.4.1), while preserving their relative differences. This approach ensures that the same track representation can be used across both the database and logical reasoning layers, creating a smooth link between data preparation, candidate selection, and the final playlist generation.

4.3 Candidate Retrieval and Filtering

Before the ASP solver constructs a final playlist, the database is reduced into a smaller set of candidate tracks. Passing the entire database directly into the solver would be inefficient and

would make the logic program unnecessarily large, since ASP grounding scales poorly with the number of facts. Hence, the database layer is used as an initial filtering method before the remaining candidate tracks are translated into an answer set program.

At this point, the user request will have already been converted into a structured form, containing fields such as playlist size, feature ranges, artist and genre constraints, popularity limits, year preferences, duration requirements, and content-based exclusion rules. The SQLite database is then queried using these fields and suitable tracks are retrieved based on the request. Candidate retrieval and filtering therefore acts as a bridge between the raw dataset and logic-based selection.

The filtering process mainly utilises the numerical audio features discussed in Section 4.2.2. The feature ranges for valence, energy, and tempo are always provided to the database query, while the further ranges such as danceability and acousticness are applied only when available. This allows the system to select tracks that approximately match the requested prompt before any kind of ASP reasoning takes place.

The database query can also apply filters using track metadata. These include: inclusion and exclusion of artists, albums, tracks and genres; year and decade constraints; popularity limits; track duration; and content-based exclusions such as explicit tracks, live versions, remixes and Christmas songs. SQLite can efficiently handle these straightforward constraints without the need for logical reasoning.

To illustrate this, consider a request for a “happy energetic playlist”. If this is interpreted as approximate feature ranges of valence in $[0.7, 1.0]$, energy in $[0.6, 1.0]$ and tempo in $[110, 150]$ BPM, querying the database with these ranges substantially reduces the set of tracks passed to the ASP solver, since only a small proportion of the dataset typically falls within these three ranges simultaneously.

When a requested genre is not present in the dataset, or there are no matches within the strict feature ranges derived from the request, two fallback steps prevent an empty result. If a requested genre cannot be found, it is removed from the query, and the user is informed via a generation note. If a genre is present, but there are no matches with the derived feature ranges, then the query is loosened using broader ranges. This makes the candidate retrieval stage more robust to tightly constrained requests.

4.4 Answer Set Programming Solver

The core reasoning technology of IntelliPlaylist is the use of Answer Set Programming to encode playlist requirements as logic program constructs. After candidate retrieval has reduced the database to a relevant set of tracks, the generated ASP program is passed to the Clingo answer set solver, as introduced in Section 2.4.2. The solver is used to evaluate potential playlist configurations and return a playlist that satisfies the encoded hard constraints, with preferences over mood, energy or artist diversity expressed through soft constraints that are described later in the section.

4.4.1 Converting tracks to ASP facts

In the logic stage, each candidate track passed into the solver is first converted into a fact-based representation. Any data retrieved from the SQLite database is expressed in a form that can be reasoned over by the ASP solver. The main fact representing each track is the `song(...)` fact, which contains the track’s unique identifier, audio feature values and track metadata.

A key design consideration was determining how to represent a track in a form that is compact enough and preserves all the information needed for playlist selection. The chosen approach is to encode each track as a single `song(...)` fact. This keeps the generated logic program simple, but still allows constraints and optimisation rules to refer to individual track features. Unlike higher-level languages and database systems, ASP does not directly support floating-point values. However, many of the selected musical features, such as energy, valence, and danceability are expressed as real numbers in the range 0.0 to 1.0. To make these values compatible with the arithmetic supported by the ASP solver, while preserving the relative differences between tracks, they are scaled to an integer in the range 0 to 100. Tempo is already measured as a numerical BPM value, so it is simply rounded to the nearest integer before being written to the ASP program.

Example 1 demonstrates this representation in greater detail. Consider the request “happy energetic playlist”. After the natural-language has been interpreted, translated into suitable feature ranges, and used to retrieve candidate tracks, assume that two matching songs remain. Suppose one track has an ID 101, tempo 118.4, energy 0.82, valence 0.76, danceability 0.79, acousticness 0.21, instrumentality 0.05, liveness 0.12, and speechiness 0.04. Under the chosen encodings, this is represented in Equation (4.1). A second candidate track with lower energy and valence could be represented in Equation (4.2).

$$\text{song}('101', 118, 82, 76, 79, 21, 5, 12, 4, 'Track A', 'Artist A', 'Album A'). \quad (4.1)$$

$$\text{song}('102', 95, 34, 40, 51, 73, 80, 10, 3, 'Track B', 'Artist B', 'Album B'). \quad (4.2)$$

In addition to the `song(...)` structure, the system also writes `performer(...)` facts as seen in Equation (4.3). This links each artist to their track identifier, which makes later artist constraints easier to express in the logic program. Multi-artist tracks are split into individual artists, producing one `performer(...)` fact per artist, so artist constraints can match any credited artist rather than only the raw artist string. The ASP input also includes target feature facts which are calculated from the midpoint of the feature ranges derived from the user request and used to represent the intended musical profile of the playlist. For instance, if the request in Example 1 produces a valence midpoint of 80, an energy midpoint of 85, and a tempo midpoint of 135 BPM, these would be represented as shown in Equations (4.4) to (4.6).

$$\text{performer}('Artist A', '101'). \quad (4.3)$$

$$\text{target_valence}(80). \quad (4.4)$$

$$\text{target_energy}(85). \quad (4.5)$$

$$\text{target_tempo}(135). \quad (4.6)$$

In addition to track and target feature facts, the system also writes request-specific facts: `required_artist(...)`, `banned_artist(...)`, `required_album(...)`, `banned_album(...)`, `required_track(...)` and `banned_track(...)`. These facts represent explicit constraints obtained directly from the user request. They do not directly select tracks themselves, but provide information that later ASP rules use to accept, reject or prefer possible playlists.

4.4.2 ASP Rules and Solver

Once the candidate tracks have been encoded as facts, the ASP rules define how a valid playlist may be selected. From the reduced candidate set produced during the filtering stage, the solver must select n tracks, where n is the target playlist size. Equation (4.7) defines the basic choice rule used for playlist selection. This allows any candidate track represented by a `song(...)` fact to be selected into the playlist as an `in_playlist` atom. However, Equation (4.7) only defines which tracks are eligible for selection, and does not control the number of tracks in the playlist. A separate hard constraint is therefore used to reject any answer set where the number of selected tracks is not equal to n , as shown in Equation (4.8). Together, they ensure that every valid playlist contains exactly the requested number of tracks.

$$\{ \text{in_playlist}(\text{ID}) : \text{song}(\text{ID}, -, -, -, -, -, -, -, -, -, -, -) \}. \quad (4.7)$$

$$:- \text{\#count} \{ \text{ID} : \text{in_playlist}(\text{ID}) \} \neq n. \quad (4.8)$$

The generated facts file and the ASP rules file are then loaded into Clingo, which grounds the program, searches for an answer set, and returns selected track identifiers as `in_playlist(ID)` atoms. These identifiers are then used to retrieve the corresponding track rows from the database. In order to keep response times acceptable for interactive use, the solver is executed with a timeout and returns the first valid playlist that it finds rather than searching the full space. A randomised seed and partial random branching are also used, so that repeated runs of the same request can produce different valid results. Because these values are configurable, a batch or evaluation context could safely use a higher timeout where latency is less critical, allowing the solver to perform a more thorough search.

4.4.3 Constraint Types and Optimisation

The ASP program uses several different types of constraints to determine what qualifies as a valid playlist. These constraints allow the system to represent clear user requirements, such as required or banned artists, and to assist the solver towards finding tracks that better match the intended playlist feeling.

The first constraint type was previously shown in Equations (4.7) and (4.8). The solver can choose from any candidate track, but any answer set that does not contain exactly n tracks is rejected. This ensures that the final playlist is the correct size, and always contains the requested number of songs.

The ASP program also employs artist based constraints. For instance, if the user requests that an artist should be excluded, a `banned_artist(...)` fact is generated. This results in any playlist containing a track that is matched to that banned artist being rejected, as shown in Equation (4.9), while a `required_artist(...)` fact rejects any playlist where no selected track matches that artist, as shown in Equation (4.10).

$$\text{:- banned_artist}(A), \text{in_playlist}(\text{ID}), \text{artist_match}(A, \text{ID}). \quad (4.9)$$

$$\text{:- required_artist}(A), 0 = \#count \{ \text{ID} : \text{in_playlist}(\text{ID}), \text{artist_match}(A, \text{ID}) \}. \quad (4.10)$$

The system can also require a minimum number of tracks from a requested artist when a user specifies to do so. This is useful when a user wants a playlist with multiple songs from the same artist, rather than an artist only appearing once. Equation (4.11) finds candidate tracks that match a required artist, while Equation (4.12) rejects any playlist where the number of selected requested-artist tracks is below the threshold amount.

$$\begin{aligned} &\text{artist_requested_track}(\text{ID}) \text{ :-} \\ &\quad \text{required_artist}(A), \\ &\quad \text{artist_match}(A, \text{ID}). \end{aligned} \quad (4.11)$$

$$\begin{aligned} &\text{:- artist_target_count}(M), \\ &\quad \#count \{ \text{ID} : \text{in_playlist}(\text{ID}), \text{artist_requested_track}(\text{ID}) \} < M. \end{aligned} \quad (4.12)$$

The same pattern is applied for album and track constraints, producing banned and required variants in each case.

Tracks that match a required artist, album, or track are recorded as protected candidates. This is used by the optimisation rules described later in this section, so that a track included to satisfy an explicit user request is not penalised for being away from the target moods or energy. This preserves user-specified content as a hard constraint without interfering with the soft constraints used to rank everything else.

Artist repetition constraints are represented in the ASP program to avoid over-repetition. If a maximum number of tracks per artist is specified then the solver counts how many selected tracks belong to each performer. If a playlist contains tracks from an artist more times than the allowed maximum, it is rejected, as shown in Equations (4.13) and (4.14).

$$\begin{aligned} &\text{artist_track_count}(A, N) :- \\ &\quad \text{performer}(A, -), \\ &\quad N = \#count \{ ID : \text{in_playlist}(ID), \text{performer}(A, ID) \}. \end{aligned} \tag{4.13}$$

$$\begin{aligned} &:- \text{max_tracks_per_artist}(M), \\ &\quad \text{artist_track_count}(A, N), \\ &\quad N > M. \end{aligned} \tag{4.14}$$

The ASP program also introduces a notion of optimisation in addition to these hard constraints. The first optimisation rule discourages repeated artists. Equation (4.15) gives a penalty when an artist appears more than once in the selected playlist. This penalty increases as the number of songs by the same artist increases.

$$\begin{aligned} &\text{repeat_artist_penalty}(A, P) :- \\ &\quad \text{artist_track_count}(A, N), \\ &\quad N > 1, \\ &\quad P = N - 1. \end{aligned} \tag{4.15}$$

The target feature facts introduced in Equations (4.4) to (4.6) represent the intended vibe of the playlist. The solver calculates how far each track is from these target values and uses penalty rules to express this distance. For example, Equations (4.16) and (4.17) calculate the absolute distance between a song's valence value and the target valence. Equation (4.16) is used when the song's valence is greater than or equal to the target, and Equation (4.17) is used when the song's valence is below the target. Tracks marked as protected candidates are excluded from these penalties, since their inclusion was already required as a hard constraint. The solver uses the same approach for energy and tempo, calculating how far each track is from the target energy and tempo values.

$$\begin{aligned}
&\text{valence_penalty}(\text{ID}, \text{D}) :- \\
&\quad \text{in_playlist}(\text{ID}), \\
&\quad \text{not protected_candidate}(\text{ID}), \\
&\quad \text{song}(\text{ID}, -, -, \text{V}, -, -, -, -, -, -, -), \\
&\quad \text{target_valence}(\text{T}), \\
&\quad \text{V} \geq \text{T}, \\
&\quad \text{D} = \text{V} - \text{T}.
\end{aligned} \tag{4.16}$$

$$\begin{aligned}
&\text{valence_penalty}(\text{ID}, \text{D}) :- \\
&\quad \text{in_playlist}(\text{ID}), \\
&\quad \text{not protected_candidate}(\text{ID}), \\
&\quad \text{song}(\text{ID}, -, -, \text{V}, -, -, -, -, -, -, -), \\
&\quad \text{target_valence}(\text{T}), \\
&\quad \text{V} < \text{T}, \\
&\quad \text{D} = \text{T} - \text{V}.
\end{aligned} \tag{4.17}$$

These penalties are then used in the optimisation statements shown in Equations (4.18) to (4.21). They are designed to guide the solver towards the playlists with the smaller overall penalties using `#minimise` statements. The priority values show the relative importance of each optimisation statement. Highest priority is given to the repeated artist penalty, as repeated artists are the most noticeable form of unwanted similarity in a generated playlist. Among the feature penalties, valence has the highest priority because it most directly captures the affective character that users describe in a mood-based request (e.g. ‘happy’, ‘sad’, ‘chill’). Energy is given the second highest priority, as it captures the perceived intensity, and tempo is weighted lowest because it acts as a harsher indicator of musical character, whose contribution is already partly captured by the energy term.

$$\text{\#minimize } \{ \text{P@4, A : repeat_artist_penalty}(\text{A}, \text{P}) \}. \tag{4.18}$$

$$\text{\#minimize } \{ \text{D@3, ID : valence_penalty}(\text{ID}, \text{D}) \}. \tag{4.19}$$

$$\text{\#minimize } \{ \text{D@2, ID : energy_penalty}(\text{ID}, \text{D}) \}. \tag{4.20}$$

$$\text{\#minimize } \{ \text{D@1, ID : tempo_penalty}(\text{ID}, \text{D}) \}. \tag{4.21}$$

Finally, the ASP program specifies that the solver output must only return selected tracks as shown in Equation (4.22).

#show in_playlist/1. (4.22)

4.4.4 Post-processing and Practical Handling

After Clingo has finished, the backend receives the final selected tracks as `in_playlist(ID)` results. However, these identifiers are not yet the final playlist shown to the user. They only refer to the selected track IDs, and do not contain the full track information. Therefore, the backend must use these identifiers to retrieve the corresponding track rows from the database, including their metadata and audio feature values.

Additional handling is required once a valid set of tracks has been selected, before the playlist can be returned to the user interface. This includes checking that enough tracks have been selected, applying any fallbacks if the solver does not return a usable result, and ordering the selected songs to give the playlist a smoother outcome.

If the original user request is too restrictive then the system includes handling to prevent playlist generation from completely falling apart. If the strict version of the request does not return enough candidate tracks, the system relaxes some of the more flexible conditions in turn, producing a more usable playlist. This is typically the popularity bounds first, then the feature ranges. When this occurs, the interface returns a generation note to indicate that the request was relaxed and which constraint was affected. If the relaxed request produces no results, a final fallback selects the candidate tracks closest to the target feature profile rather than failing outright.

Finally, the playlist ordering is performed post-ASP selection. The ordering algorithm is greedy and nearest-neighbour. The first track is chosen as the candidate which is closest to the target feature profile, and then each subsequent track minimises a transition cost relative to the previous track. This transition cost is a weighted sum of the absolute differences across tempo, energy, valence, danceability, acousticness, instrumentalness, liveness, and speechiness. Energy and valence are weighted because they correspond to arousal and valence, the two principal axes of Russell’s circumplex model of affect [24]. They are also the audio-feature descriptors linked with perceived mood in music emotion recognition [30]. Smoothing transitions on these features helps preserve a consistent theme across consecutive tracks. An artist cooldown penalty is then applied when the candidate’s artist matches one of the most recently chosen tracks. This spaces out songs with the same artist in the playlist’s listening order without removing them from the playlist. This means that the solver selects the content, while post-processing prepares the playlist for presentation to the user.

4.5 Natural-Language Request Interpretation

This section presents the method used to convert natural-language playlist descriptions into the structured constraint representation used by the candidate retrieval and ASP generation stages described in Sections 4.3 and 4.4.

The approach uses a large language model (LLM) to convert the user’s description into a structured intermediate request, which the backend then translates into database filters, ASP facts, and solver constraints. The LLM is therefore responsible only for the natural language interpretation, while the backend remains responsible for producing a logic program that follows the encoding described in Section 4.4.

4.5.1 Using LLMs to Convert Natural Language to a Structured Playlist Request

The structured request is represented as a JSON object that captures the user’s intent in a fixed, machine-readable format. To make this reliable, the LLM is constrained to return JSON that conforms to a fixed schema. This is done by using OpenAI’s Structured Outputs feature, which accepts a schema definition expressed as a Pydantic model and ensures that the model’s response will match it. The schema includes a closed vocabulary of fourteen emotion labels over which the LLM is expected to distribute weights for a given request. These labels are joy, energetic, calm, focus, sad, anger, romantic, nostalgic, confident, dark, dreamy, hopeful, tense, and sensual.

The parser prompt given to the model instructs it to return JSON only, using fields such as `original_text`, `emotions`, `emotion_weights`, `activity`, `playlist_size`, `artist constraints`, `genre constraints`, `popularity constraints`, `duration constraints`, `exclusion rules` and `audio-feature ranges`. It also explains how any missing values should be represented, using `null` for missing single values, empty lists for missing list fields, and empty objects for missing emotion weights. The prompt also gives the model various interpretation rules for different types of playlist language. It explains how emotion and activity words should be converted into emotion weights, how artist specific requests should be placed into `include` and `exclude` fields, and how duration wording can differ between total playlist duration and individual track duration. There are also instructions for explicit audio-feature requests, such as interpreting “high energy” or “no vocals”.

The conversion from these emotion weights and activity information into audio feature ranges is handled by the backend, rather than by the LLM itself. The backend maintains a mapping from each emotion label to a profile of ranges across audio features described in Section 4.2.2. Each profile expresses the typical musical character of that emotion. For example, joy covers high valence (0.70–1.00), moderate-to-high energy (0.55–0.95) and a fast tempo range (105–150 BPM), and focus covers moderate valence (0.30–0.65), low-to-moderate energy (0.20–0.50) and a slower tempo (65–115 BPM). If a request were to contain multiple emotions, the corresponding profiles are combined by a weighted average to produce final ranges that are then passed to the candidate retrieval stage. By keeping the mapping in the backend, it means that the same emotion produces the same feature ranges across runs, and is also straightforward to inspect or modify.

This division of responsibility means the LLM only interprets the user’s natural-language description, while the backend converts the structured request into filters, facts and constraints. This retains consistency within the final ASP program through the use of the encoding described

in Section 4.4, while still allowing the user to describe the playlist naturally.

4.5.2 Parser Fallback and Pipeline Integration

Once the JSON request described in Section 4.5.1 has been passed into the Python backend, it is used to guide the later stages of the playlist generation pipeline. Each field within the request is used to guide candidate track retrieval from the SQLite database. This includes constraints such as included artists, popularity limits, and the filtering out of explicit content. This same structured request is then also used when generating ASP facts, such as target feature values and any directly requested constraints.

The system also includes a fallback to improve its overall robustness. If the OpenAI API cannot be reached, returns an error, or produces a response that fails schema validation, the rule-based parser is used instead. This prevents the system from becoming solely reliant on the LLM response being successful. The fallback parser uses keyword matching and regex extraction over a fixed vocabulary, allowing it to identify mood words, activity terms, playlist size, artist inclusions and exclusions, and direct audio-feature requests such as “high energy”. It cannot capture nuanced or unfamiliar mood descriptions, multi-clause requests, or constraints phrased indirectly, and so produces a noticeably lower-resolution interpretation than the LLM parser when it is invoked.

4.6 Web Application and User Interface

In the final stage of development, a web user interface (UI) is developed to demonstrate the playlist generation pipeline described in Sections 4.2 to 4.5 in a usable and interactive setting. The interface allows a user to enter a natural-language request, view the structured constraints that have been extracted from it, and receive a final playlist that can then be exported directly to a Spotify account.

4.6.1 Application Structure

The application uses the React Framework written in TypeScript for the front end, paired with a Flask API written in Python for the back-end. This is integrated with the OpenAI API to convert natural-language requests into the structured form described in Section 4.5. The Clingo answer set solver, called as a Python library, then selects tracks that satisfy the resulting constraints. Track metadata and audio features are then retrieved from the SQLite database described in Section 4.2.1, and the Spotify web API is used to export the final playlist to the user’s Spotify account. A diagram of the application’s structure can be seen in Figure 4.2.

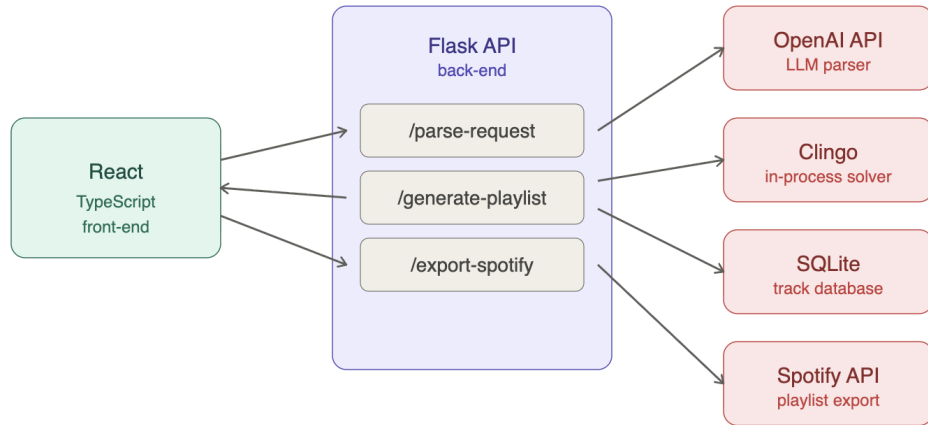


Figure 4.2: Diagram of the IntelliPlaylist application structure.

4.6.2 Front-End

The front-end of the application consists of a basic user interface (UI) that allows a user to enter a playlist in natural language and view the resulting playlist once it has been generated. The initial welcome screen presents a brief description of the system and a prominent “Ask IntelliPlaylist” button. Pressing this opens a request modal containing a text area for the user to type their description, along with a submit button that triggers parsing and generation.

Once the playlist has been generated, the result is displayed as a track list. Each track shows its title, artist, and album, along with its position in the playlist. In addition to being able to look at each track’s attributes, the user can act on individual tracks through three different controls. They can lock a track to prevent it from being changed during regeneration, they can remove a track from the playlist entirely, or they can replace a track with a different candidate that fits the same constraints. The user can also regenerate the entire playlist with the same request so that they can produce a different valid result, or they can export the playlist directly to a Spotify account through the export flow described in Section 4.6.3.

Below the playlist, the interface displays a request interpretation panel showing how the system parsed the user’s request. This includes the extracted emotion weights, activity, playlist size, ordering style, and any feature ranges or artist constraints. This allows the user to see how their request was understood before any tracks are produced, addressing the lack of transparency discussed in Section 2.1.3.

4.6.3 Back-End

This subsection covers the structure of the application’s back-end in more detail. The Flask API consists of three main endpoints (as shown in Figure 4.2) along with a supporting OAuth flow for Spotify authentication. These endpoints include **PARSE-REQUEST**, **GENERATE-PLAYLIST**, and **EXPORT-SPOTIFY**.

The PARSE-REQUEST endpoint takes a user's natural-language description and converts it into the structured playlist request described in Section 4.5.1. This is done using the OpenAI structured outputs parser with a rule-based fallback. The parsed request is required as a JSON object and is rendered in the request interpretation panel described in Section 4.6.2, allowing the user to see how their request was interpreted.

The GENERATE-PLAYLIST endpoint takes a structured PlaylistRequest and runs the candidate retrieval, ASP, and post-processing stages described in Sections 4.3 and 4.4, returning the selected tracks and any generation notes. Separating it from the parsing means that the same parsed request can be run multiple times for variety, and individual tracks can be replaced.

The EXPORT-SPOTIFY endpoint creates a Spotify playlist from the given track list. It searches for each track by name and artists, before adding the matches to a new private playlist with a name chosen by the user. Authentication is handled through the supporting OAuth login, callback, and status endpoints.

Chapter 5

Implementation and testing

This chapter describes how the design process described in Chapter 4 was implemented and tested. The system was built in Python using Flask for the back-end and React with TypeScript for the front-end, with SQLite as the track database, Clingo as the answer set solver invoked as a Python library, the OpenAI API for natural-language parsing, and the Spotify Web API for playlist export. Together, these components implement the pipeline shown in Figure 4.1.

This chapter is organised around the main stages of this pipeline, in which a user’s request is parsed into a structured intermediate, used to query a set of candidate tracks, passed through the ASP solver, post-processed, and returned to the interface for display and export. Section 5.1 covers the dataset import process and the candidate retrieval layer. Section 5.2 describes the natural-language parser, including an earlier rejected approach in which the LLM was asked to generate answer set programs directly. Section 5.3 covers the ASP solver integration and the web application. Section 5.4 then presents the functional testing carried out to evaluate the system.

5.1 Dataset and Candidate Retrieval

5.1.1 Dataset Import

The Spotify tracks dataset was loaded into SQLite using an offline Python script. The source CSV was read in chunks of 50,000 rows using pandas, with each chunk cleaned before insertion. Column names were normalised onto the database schema and any missing values in the optional fields were filled with practical replacements (empty strings for text, zero for numerical features, etc.) and any rows missing a valid track ID or name were discarded.

A few edge cases appeared during the build. Some rows had artist fields which contained semicolons used to separate multiple artists. These were preserved as raw strings at this stage and split later when generating the ASP performer facts, as described in Section 5.3. Duration values were also converted from milliseconds into seconds so that it is easier to use them later when filtering. Finally, a small number of rows had missing values in numerical

audio-feature columns. These rows were dropped rather than filled in, to avoid affecting the feature-range logic used later in the system.

The final database holds 114,000 tracks across 125 genres in a single tracks table. Indexes were created over the name, album, artists, genre, popularity, and year columns to support the candidate retrieval stage.

5.1.2 Candidate Retrieval

The candidate retrieval stage is implemented as a single constructed SQL query against the tracks table. It reads the parsed request field by field, and each non-null field is added to the query. Feature ranges become BETWEEN predicates, artist and genre inclusions and exclusions become IN/NOT IN filters, popularity and year bounds become inequalities, and the content-based exclusions (such as explicit tracks, live versions, remixes and Christmas songs) are handled by checking the track name with LIKE filters.

The query uses parameter binding instead of directly joining the strings together. This helps prevent SQL injection, especially because some values come from the parser. A final LIMIT is also added to cap the number of candidate tracks that are returned. This keeps the later ASP stage manageable, even when the user gives a very loose request.

The two fallback steps that were described in Section 4.3 are also implemented at this layer. If a requested genre is not present in the database, the genre filter is dropped and a note is added to the generation response. If a more strict query ends up resulting in fewer candidates than the requested playlist size, the feature ranges are widened and the query is re-run.

In practice, this filtering layer reduces the dataset substantially before calling the ASP stage. The row limit is set dynamically as $\max(100, \text{track_count} * 10)$, so a 10-track request passes 100 candidates to the solver and a 20-track request passes 200. For the example used in Chapter 4, “happy energetic playlist”, the strict query returns around 3,500 candidates. Any tighter requests with additional constraints (specific genre, popularity bounds, etc.) may have fewer than the limit available to sample from, in which case all matching rows are returned. Looser requests can match 8,000 or more rows, but the limit is always the binding factor on what the ASP solver sees.

5.2 Natural-Language Parser

This section describes the natural-language parser as it was built. Two approaches were investigated during development. The first, which proved unreliable and was rejected, was to have the LLM generate the answer set program directly from the user’s request. The second approach, used in the final system, uses the LLM to produce a structured intermediate request. This was described in Section 4.5.1 of the Design chapter.

5.2.1 Using LLMs to Convert Natural Language to ASP

A large language model's ability to generate an answer set program from a natural-language playlist request was first explored. The GPT-4o model was used in this process. Other options, such as Google's Gemini [10] and a number of models available from Hugging Face [12], were also considered. However, the capabilities and results of the OpenAI API proved superior for integration with IntelliPlaylist.

Figure 5.1 shows an attempt to generate an answer set program for the playlist request from Example 1 in Chapter 4. The program generated by the LLM can be seen in Figure 5.2. However, it does not follow IntelliPlaylist's predicate structure, instead using predicates such as `playlist_song(...)`, `happy_song(...)` and `valence(...)`. It also uses floating-point values such as 0.7 and 0.75, whereas IntelliPlaylist scales audio-feature values to integers before passing them to ASP. As a result, the generated program cannot be used by the existing backend pipeline.

```
Generate an answer set program to specify constraints
for a playlist request. The playlist request is "happy
energetic playlist". The playlist should contain 10 songs.
It should prefer tracks with a happy and energetic mood,
meaning tracks with high valence and high energy. The
program should select songs from the available candidate
tracks and return the selected playlist.
```

Figure 5.1: Example of prompt given to LLM to generate ASP from playlist description

```

% Facts describing the requested playlist
playlist_size(10).
mood(happy).
mood(energetic).

% Select songs for the playlist
10 { playlist_song(Song) : track(Song) } 10.

% A happy song should have high valence
happy_song(Song) :-
    valence(Song, V),
    V > 0.7.

% An energetic song should have high energy
energetic_song(Song) :-
    energy(Song, E),
    E > 0.75.

% All selected songs should match the requested mood
:- playlist_song(Song), not happy_song(Song).
:- playlist_song(Song), not energetic_song(Song).

% Optimise towards high energy and valence
#maximize { V + E : playlist_song(Song), valence(Song, V), energy(Song, E) }.

#show playlist_song/1.

```

Figure 5.2: Program generated by LLM following prompt in Figure 5.1

One-Shot Learning was then used, where the model is given an example in an attempt to familiarise it with the problem at hand and thus improve the quality of the solution. An example playlist description was given to the model and translated into an answer set program. The prompt used for this can be seen in Figure 5.3.

Generate an answer set program to specify constraints for a playlist request.
 Here is an example playlist request:
 "calm study playlist, 8 songs"

This should be translated into the following answer set program structure:
 song("101", 80, 25, 45, 40, 70, 30, 10, 5, "Track A", "Artist A", "Album A").
 song("102", 90, 30, 50, 35, 60, 45, 12, 4, "Track B", "Artist B", "Album B").

```
target_valence(45).
target_energy(30).
target_tempo(85).
```

```
{ in_playlist(ID) : song(ID, _, _, _, _, _, _, _, _, _, _, _) }.
:- #count { ID : in_playlist(ID) } != 8.
```

```
#minimize { D@3, ID : valence_penalty(ID, D) }.
#minimize { D@2, ID : energy_penalty(ID, D) }.
#minimize { D@1, ID : tempo_penalty(ID, D) }.
```

```
#show in_playlist/1.
```

Now generate an answer set program for the following playlist request:
 "happy energetic playlist"

Figure 5.3: One-shot learning prompt given to LLM to generate ASP from playlist description

```
% Candidate tracks for happy energetic playlist
song("201", 128, 90, 88, 82, 12, 5, 15, 6, "Happy Song", "Artist X", "Album X").
song("202", 135, 86, 80, 78, 10, 4, 12, 5, "Energy Track", "Artist Y", "Album Y").

target_valence(85).
target_energy(90).
target_tempo(130).

{ in_playlist(ID) : song(ID, _, _, _, _, _, _, _, _, _, _, _) }.
:- #count { ID : in_playlist(ID) } != 10.

#minimize { D@3, ID : valence_penalty(ID, D) }.
#minimize { D@2, ID : energy_penalty(ID, D) }.
#minimize { D@1, ID : tempo_penalty(ID, D) }.

#show in_playlist/1.
```

Figure 5.4: Program generated by LLM following prompt in Figure 5.3 when queried

Figure 5.4 shows the program generated by the LLM using the one-shot learning prompt in Figure 5.3. Compared with the earlier response in Figure 5.2, this was an improvement because it follows more of the IntelliPlaylist structure by using song(...), target_valence(...), target_energy(...), target_tempo(...) and in_playlist(ID). However, this does not come without

its drawbacks. The model generates its own candidate song(...) facts rather than using tracks retrieved from the database. This means that the selected IDs may not match the actual songs in the system. It also may leave out constraints, change predicate structures, and produce optimisation rules that do not match the system's backend.

As a result, this approach was discarded in favour of using the LLM to produce a structured intermediate request, as described in Section 4.5.1.

5.2.2 Using LLMs to Convert Natural Language to a Structured Playlist Request

The structured parser described in Section 4.5.1 was implemented using OpenAI's structured outputs feature, which constrains the model's response to match a Pydantic schema. This schema defines all of the fields which the IntelliPlaylist system expects, including the closed set of fourteen emotions. This means the parser returns data in a predictable format, so the backend can read it directly without needing extra checks or complex error handling.

The final prompt used by IntelliPlaylist is significantly longer than the extract shown in Figure 5.5. This is because it contains highly complex rules for interpreting mood, activity, artists, duration, popularity, audio features and exclusions. Therefore, only a shortened extract is included to demonstrate the structure of the prompt without reproducing the full implementation text. The prompt uses **Example 2** as the request: "happy energetic gym playlist, 15 songs, no Drake, clean only, popular songs". This request contains several different constraint types, which makes it useful for showing how the LLM output can represent a user's intent without directly generating ASP.

```
SYSTEM: You convert playlist requests from natural language into structured JSON.

Return JSON only with these keys:
original_text, emotions, emotion_weights, activity, constraint_explanation,
playlist_size, duration_minutes, include_artists, exclude_artists,
include_genres, exclude_genres, min_popularity, max_popularity,
exclude_explicit, tempo_range, energy_range, valence_range, ordering_style ...

Rules:
- Keep original_text exactly as given.
- Only fill values that are explicit or strongly implied by the user.
- Use null for missing single values.
- Use [] for missing lists.
- Use {} for missing emotion_weights.
- Do not invent artists, albums, tracks, genres, years, durations or ranges.

USER: happy energetic gym playlist, 15 songs, no Drake, clean only, popular songs
```

Figure 5.5: Extract from the prompt given to the LLM to produce a structured playlist request.

```

{
  "original_text": "happy energetic gym playlist, 15 songs, no Drake, clean only, popular songs",
  "emotions": ["happy", "energetic"],
  "emotion_weights": {
    "joy": 0.4,
    "energetic": 0.6
  },
  "activity": "workout",
  "constraint_explanation": "Interpreted as a happy, energetic gym playlist with 15 popular clean tracks and Drake excluded.",
  "playlist_size": 15,
  "duration_minutes": null,
  "include_artists": [],
  "exclude_artists": ["Drake"],
  "include_genres": [],
  "exclude_genres": [],
  "min_popularity": 60,
  "max_popularity": null,
  "exclude_explicit": true,
  "tempo_range": null,
  "energy_range": null,
  "valence_range": null,
  "ordering_style": "smooth"
}

```

Figure 5.6: Structured playlist request produced from the LLM prompt in Figure 5.5.

Figure 5.6 shows how the LLM extracts the important parts of the user’s request into a structured JSON format that can be processed by the backend. The request is therefore split into fields that can be handled consistently throughout the system pipeline. For example, “15 songs” is stored as `playlist_size`, “no Drake” is stored in `exclude_artists`, “clean only” is represented using `exclude_explicit`, and “popular song” is represented using `min_popularity`. The emotions, `emotion_weights` and `activity` fields provide a link between vague natural-language requests and the numerical audio feature values that are used in the database. In Figure 5.6, the emotions are interpreted as having both joyful and energetic attributes and “gym” is mapped to the activity workout.

5.3 ASP Solver and Web Application

5.3.1 Clingo Integration

The Clingo solver is run directly from Python using the official Clingo library. This lets the system load the ASP files, ground the program, and perform the solve within the same Python process. Because of this, the backend does not need to start a separate command-line process for every request.

The `clingo.Control` object is constructed with several command-line equivalent options. The playlist size is passed using `-c n=N`, so the ASP program knows how many tracks to select. Two other options are used, affecting how the solver searches. These are `--opt-mode=ignore` and `--models=1`, which tell Clingo to ignore optimisation statements and return the first valid playlist it finds, instead of searching the full space for an optimal one.

In the end, ignoring optimisation statements was essential in order to maintain response times. With optimisation enabled, the solver could spend minutes searching for an optimal model of candidate tracks of a few hundred tracks. By utilising `--opt-mode=ignore`, the first satisfying model is returned almost immediately. Consequently, the quality of the final

solution is determined by candidate retrieval, which has already filtered the dataset. The `#minimize` statements have still been retained in the rules file so that the program remains consistent and so that optimisation could still be re-enabled for future experiments.

In order to prevent the first valid playlist from being the exact same every time, two extra options are used to add randomness. `--rand-freq=0.25` enables random branching with 25% probability, and `--seed` is set to a fresh random integer on each call. Alongside the `ORDER BY RANDOM()` step in candidate retrieval, this means the same user request can produce different valid playlists on different runs.

The solver is also given a five-second timeout. If the solver completes within the timeout, the backend uses the model returned by Clingo. If it does not finish within the timeout, the solver is cancelled, and the system falls back to the simpler candidate selection method described in Section 4.4.4. In practice, the timeout is rarely reached for typical requests, but it prevents the system getting stuck when a request is too strict or impossible to satisfy.

5.3.2 ASP Encoding Considerations

Generating the facts file from Python requires handling three issues that the ASP rules in Section 4.4 take for granted.

Float-to-integer scaling. Clingo only supports integer arithmetic, so the audio features stored in the database as real numbers in $[0.0, 1.0]$ are multiplied by 100 and rounded before being written into the song/12 fact. Tempo is already stored in BPM, so it only needs to be rounded. None values are changed to 0 to avoid writing a `None` literal into the ASP file which would create invalid syntax.

String escaping. Track, artist, and album names can sometimes contain characters such as double quotes and backslashes. These have to be escaped before they can be written into the ASP file. A small helper function achieves this by replacing `\` with `\\` and `"` with `\`. Without this step, some track names like `Divertimento No. 11 in D, K.251` ‘Nannerl-Septett’ would break the ASP syntax and cause Clingo to fail.

Multi-artist handling. The Spotify dataset stores multi-artists in a few different ways. Sometimes the artists are stored as a stringified Python list, such as `['Artist A', 'Artist B']`. Other times they are separated using semicolons, such as `Artist A;Artist B`, or sometimes commas. To handle this, a `split_artists` helper is used; This tries to read each value as a list, then falls back to splitting on semicolons or commas. For each artist found, a separate performer/2 fact is written linking that artist to the track ID.

5.3.3 Web application

The Flask back-end provides the three endpoints listed in Section 4.6.3, as well as supporting endpoints for the OAuth flow. These include `/api/spotify/login`, `/api/spotify/callback`, `/api/spotify/status`, and `/api/health`. CORS is also configured so that the local React front-end can send requests to the Flask back-end during development.

The OAuth flow uses Spotify’s standard authentication-code process. When the user starts

logging in, a `state` value is generated using `secrets.token_urlsafe(24)` and sent with the Spotify login request. This value is then checked again when Spotify redirects back to the application, helping to protect against any cross-site request forgery. Spotify access tokens are stored in the Flask session with their expiry time. When `/api/export-spotify` is called, the back-end checks if the token is valid. If it has expired, the refresh token is used to get a new access token before continuing. If there is no token at all, the endpoint returns 401 with an `auth_url`, which the front-end uses to start the login process.

The front-end is implemented in React with TypeScript. Figure 5.7 shows the initial view, while Figure 5.8 shows a request modal which opens when the user presses the “Ask IntelliPlaylist” button. On submission, the front-end calls `/api/parse-request` and then `/api/generate-playlist` in sequence, and renders the resulting playlist and an expandable interpreted request panel, as shown in Figures 5.9 and 5.10.

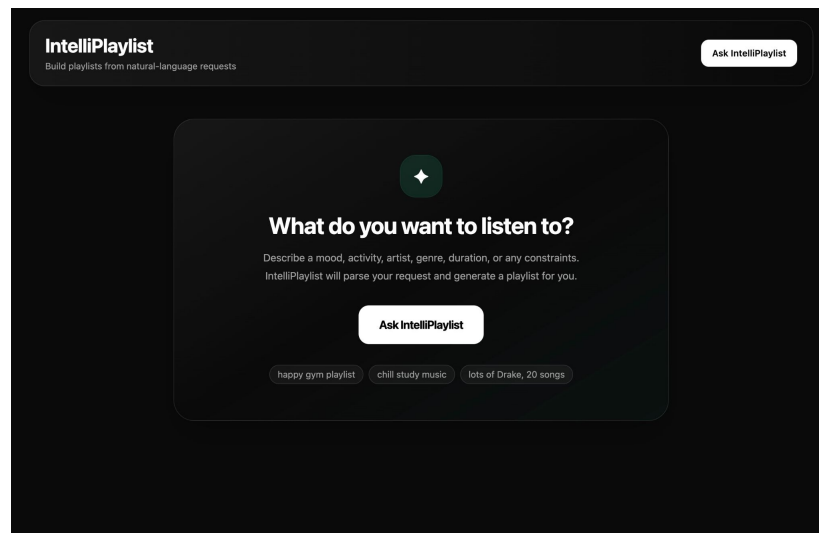


Figure 5.7: Initial User Interface.

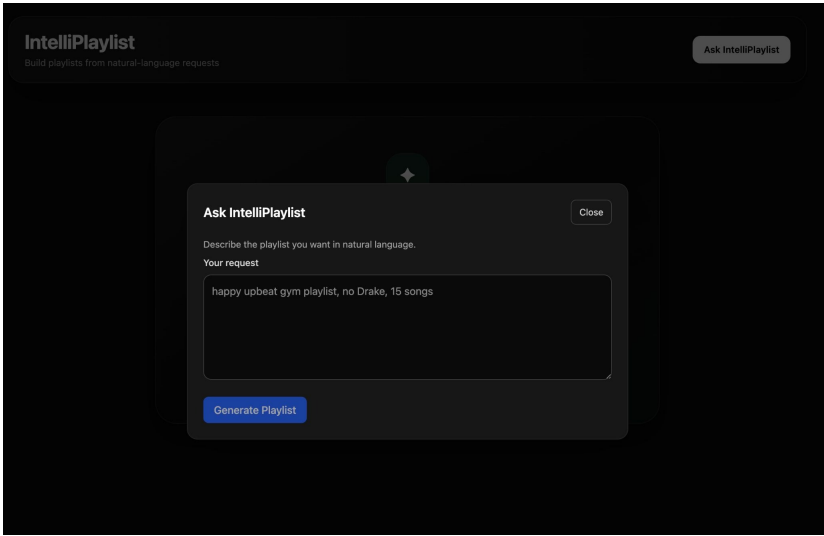


Figure 5.8: Request modal where users describe their desired playlist in natural language.

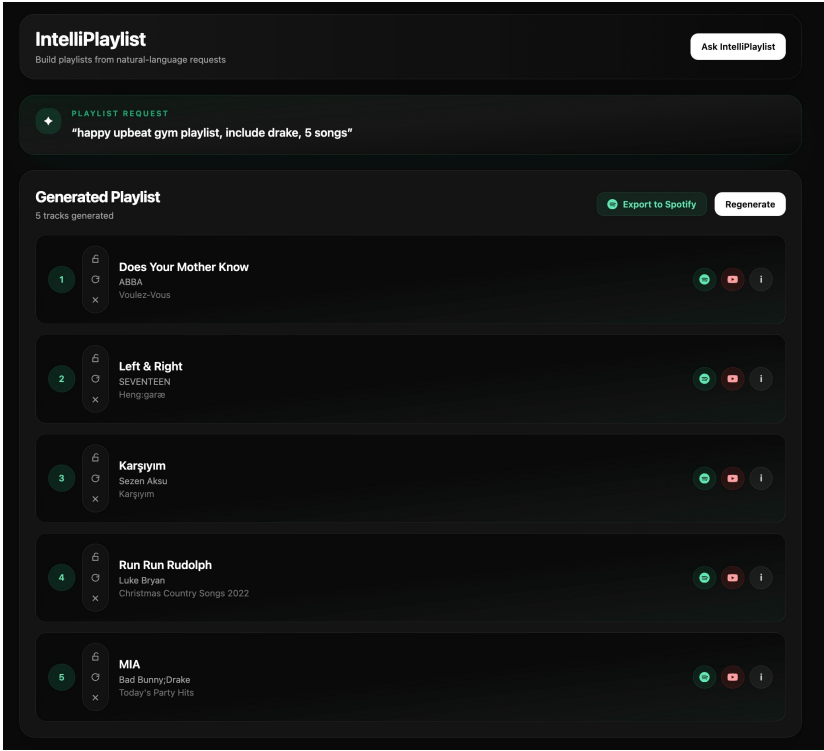


Figure 5.9: Generated Playlist View.

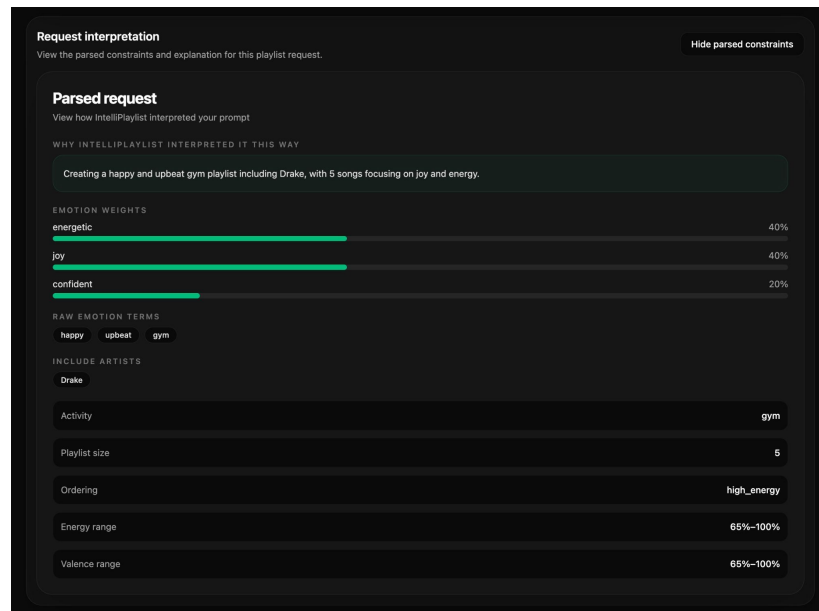


Figure 5.10: Structured playlist request from the description used in Figure 5.9.

5.4 Functional Testing

Functional testing was carried out using the category-partition approach described in Section 3.4. The four main stages of the pipeline were each tested using a small set of representative inputs. These stages included the natural-language parser, candidate retrieval, ASP solver, and post-processing. Each set was chosen to cover both normal examples, and then at least one edge case that tests the component in more difficult situations. The aim was to confirm that each stage of the system behaves as the Design chapter says that it should, and to surface any case where it does not.

Each test below records the given input to the component, the behaviour that was expected, the behaviour that was observed, and whether the test was passed.

5.4.1 Natural-language parser

The natural-language parser converts the free-text input request into a `PlaylistRequest` object. The tests in this section confirm that mood, activity, playlist size, artist constraints, and content filters can be correctly identified. They also check that the parser can handle nonsense input without crashing.

Table 5.1: Functional tests for the natural-language parser.

Test	Expected	Observed	Result
Simple mood <i>“happy songs”</i>	“happy” is recognised as a joy emotion weight	emotions=[‘happy’], emotion_weights={joy: 1.0}	Pass
Mood + activity <i>“chill study music”</i>	“study” is recognised as an activity; “chill” combines with study to produce calm and focus emotion weights	activity=study, emotion_weights={calm: 0.5, focus: 0.5}	Pass
Explicit constraints <i>“15 happy songs, no Drake, clean only”</i>	The size (15), the excluded artist (Drake), and the explicit-content filter are all extracted	Drake exclusion and explicit-content filter correctly extracted; playlist_size not extracted (None)	Partial
Multi-clause <i>“upbeat gym playlist, popular only, no live versions”</i>	“gym” maps to the workout activity; “popular only” sets a minimum popularity; “no live versions” sets the live-exclusion flag	activity=workout, min_popularity=60, exclude_live=True	Pass
Gibberish input <i>“asdfgh”</i>	The parser does not raise an exception; all extracted fields are empty or default	No exception raised; all fields empty/default	Pass

The explicit constraint test was a partial pass. The playlist size was not extracted because the adjective “happy” between “15” and “songs” prevented the fallback parser’s regex from matching.

5.4.2 Candidate Retrieval

Once the structured JSON request has been produced, the candidate retrieval stage can begin. This is where an SQL query is created in order to filter the 114,000 tracks in the database down to a smaller set to be passed to the ASP solver. These tests confirm that the filtering narrows down the database as expected and that the fallback mechanisms described in Section 4.3 work when needed.

Table 5.2: Functional tests for candidate retrieval.

Test	Expected	Observed	Result
Loose request <i>Request with no specific constraint</i>	Many rows match the filter; the dynamic LIMIT cap of <code>max(100, track_count * 10)</code> becomes the binding factor	100 candidates returned (cap reached)	Pass

Continued on next page

Table 5.2 continued from previous page

Test	Expected	Observed	Result
Tight request <i>Study-style energy range, 2010s decade</i>	The tight filter narrows the candidate set substantially	0 candidates pre-fallback under the tight filter; 100 under the loose filter	Pass
Unknown genre <i>Request including “zorbcore”</i>	The system drops the unrecognised genre, returns tracks, and notifies the user via a generation note	5 tracks returned; generation note explaining the unknown genre returned to the user	Pass
Impossible tempo range <i>Tempo range 300–301 BPM</i>	No real tracks match; the system widens the tempo range via fallback rather than returning an empty playlist	Broad fallback invoked; 5 tracks returned	Pass

5.4.3 ASP solver

The tests below confirm that the hard constraints defined in the ASP rules file are enforced when the solver runs. This is important given that the solver is run with `--opt-mode=ignore`, as discussed in Section 5.3.1. This disables the `#minimize` optimisation statements but does not disable the hard constraints.

Table 5.3: Functional tests for the ASP solver.

Test	Expected	Observed	Result
Required artist <i>5-track playlist with The Beatles set as a required artist</i>	At least one Beatles track in the returned playlist	1 Beatles track present	Pass
Banned artist <i>10-track happy playlist with Drake banned</i>	Zero Drake tracks in the returned playlist	0 Drake tracks present	Pass
Playlist size <i>Request for exactly 15 chill tracks</i>	Exactly 15 tracks returned (the <code>#count != n</code> constraint rejects other sizes)	15 tracks returned	Pass
Max tracks per artist <i>Playlist with <code>max_tracks_per_artist=2</code></i>	No artist appears more than twice in the returned playlist	Maximum of 1 occurrence per artist	Pass

5.4.4 Post-processing

Once the solver has returned its playlist selection, post-processing then reorders the tracks to create a smoother listening sequence for the user and space out any repeated artists. These tests check both these behaviours.

Table 5.4: Functional tests for post-processing.

Test	Expected	Observed	Result
Artist cooldown <i>20-track playlist containing multiple tracks by the same artist, cooldown = 3</i>	No two same-artist tracks within 3 positions of each other in the final ordering	0 cooldown violations	Pass
Smooth ordering <i>10-track playlist, ordered output vs same tracks in random order</i>	The greedy nearest-neighbour ordering produces a lower mean transition cost across consecutive tracks than a random shuffle	Ordered mean cost: 1.13; random mean cost: 1.35 (mean of 5 shuffles)	Pass

The second test proved to be the most quantitative result of any of the functional tests. When consecutive tracks were compared, the average transition cost was 1.13 under greedy ordering and 1.35 with random ordering. This is a reduction of about 17%. This shows that the ordering step is a definite improvement for the playlist structure. It does more than rearrange the solver output for presentation; it makes the transition between tracks smoother in a measurable way.

5.4.5 Summary

Across all 15 tests, 14 passed and one passed partially, with the partial pass being a known limitation of the rule-based regex, rather than a defect in the pipeline. Chapter 6 evaluates the quality of the playlists produced against a random baseline.

Chapter 6

Results and discussion

The evaluation of IntelliPlaylist is split into two parts. First, the functional testing in Section 5.4 confirmed that each pipeline state worked as expected. Second, a comparison study was used to assess the quality of playlists generated by the system.

This chapter reports the findings from the comparison study and relates them back to the requirements from Section 3.1. It then discusses the project’s limitations and any possible directions for future work.

6.1 Comparison Study and Findings

Ten unique natural-language requests were used for this study. These requests covered different moods, activities, and constraint types in order to generate twenty playlists. Ten of these were produced via IntelliPlaylist, and ten were produced using a random baseline. The random baseline simply selected tracks randomly from the database using:

```
SELECT id, name, artists FROM tracks ORDER BY RANDOM() LIMIT N. (6.1)
```

Each playlist was rated without showing whether it came from IntelliPlaylist or the baseline. The playlists were assessed using three measures: precision, novelty, and subjective quality. Precision measured how many tracks matched the request, novelty measured how many tracks were unfamiliar, and subjective quality gave each playlist an overall quality score from 1 to 5. The full per-playlist ratings are given in Appendix A.1.

Table 6.1: Mean ratings across the ten requests for each measure.

Metric	IntelliPlaylist	Random Baseline	Difference
Precision	0.62	0.46	+16pp
Novelty	0.65	0.65	0pp
Quality (1–5)	3.10	2.20	+0.90

In a head-to-head comparison across the ten requests, IntelliPlayist performed better than the random baseline in most cases. It had higher precision on 7 out of 10 requests and higher subjective

quality on 6 requests, with 3 ties and 1 loss. Novelty was much closer. Intelliplaylist had higher novelty in 5 cases, tied in 2 and lost 3. This suggests that the system produces playlists that are noticeably more relevant and higher-quality than random selection, while still suggesting unfamiliar tracks.

The clearest failure was the **nostalgic 80s pop request**, which scored 0/10 on precision and 1/5 on quality. Further inspection showed that the dataset contains no tracks with 1980s year values, so the strict pipeline returns an empty playlist for any era-specific 80s request (See Appendix A.2.2). The playlist used in the evaluation was therefore created using a manual fallback, where the decade constraint was removed. This shows a genuine limitation of the dataset, surfaced by the evaluation rather than the functional tests.

Two more defects appeared while generating and rating the playlists. Firstly, duplicate tracks appeared in two playlists 12 and 18, as shown in Appendix A.2.1. On inspection, these were not duplicate database rows but rather the same recording listed multiple times under different album titles. For example, All The Small Things by blink-182 appears in the dataset under several albums, including 90's Music Grandi Successi, Halloween Kids Party 2022, and Rock Workout 2023. Because the system treats each row as a separate track, the same song can be selected more than once. This issue is readily addressed in post-processing and could be fixed by removing duplicates based on track name and artist before returning the playlist. The second defect was more significant. As shown in Appendix A.2.2, the system returns an empty result when the candidate pool is empty, but does not display a clear warning to the user. This goes against the intended design as the system should either widen the search or explain why no playlist could be generated. This is therefore a real limitation of the current implementation.

6.2 Requirements Analysis

6.2.1 Functional Requirements

All of the functional requirements were achieved but the “avoid generating playlists with duplicate tracks” requirement. This is discussed in Section 6.1.

Table 6.2: Success of Functional Requirements of the project.

Requirement	Pass/Fail
Implementation of natural language input interface that allows users to describe their desired playlist	Pass
Implementation of user control over playlist length	Pass
Implementation of a component that maps user descriptions to an internal emotion profile and corresponding audio features	Pass
Implementation of a playlist generation feature that uses an Answer Set Programming Solver and the dataset of tracks to select tracks relevant to the computed feature ranges	Pass
Display the generated playlist in a clear format	Pass
Avoid generating playlists with duplicate tracks	Fail
Implement the ability for users to specify multiple moods within one request	Pass

Continued on next page

Table 6.2 continued from previous page

Requirement	Pass/Fail
Display multiple playlists for the same description to allow for comparison	Pass

6.2.2 Non-Functional Requirements

All of the non-functional requirements were achieved.

Table 6.3: Success of Non-Functional Requirements of the project.

Requirement	Pass/Fail
The web app should be simple and friendly	Pass
Playlists should be generated within a short, interactive time	Pass
The web app should be responsive to a variety of screen sizes	Pass
The web app should be available across all major browsers	Pass

6.3 Challenges and Further Work

There are several future improvements that have emerged following the limitations found in the evaluation. Firstly, the dataset coverage gap could be addressed by introducing a year field to the dataset since the current Spotify tracks dataset does not include one. Secondly, the duplicate track defect could be fixed in the post-processing stage by removing songs with the same name and artists before returning the playlist. The runtime `--opt-mode=ignore` trade-off could also be addressed. In the current implementation, this improves response times by returning the first valid playlist. Future work could explore whether a bounded optimisation mode, such as `--opt-mode=opt` with a time limit (`--time-limit=N`), could improve playlist quality while keeping response times acceptable. Finally, the comparison study could be expanded. The current study used one rater and ten requests, so the results should be regarded as indicative and not statistically conclusive. A larger study with multiple independent raters, ethics approval, and a wider range of requests would give much stronger evidence for how effective the system truly is.

Chapter 7

Conclusion

The overall aim of this project was to allow users to generate personalised music playlists from natural-language descriptions, using Answer Set Programming to reason over track audio features in a structured and explainable way. Existing commercial systems such as Spotify and Apple Music operate largely as black boxes, giving users very little visibility into how recommendations are produced and a restricted ability to apply their own constraints, as discussed in Chapter 2. The functional and non-functional requirements established in Chapter 3 shaped the design and implementation to address these limitations, treating playlist generation as a constraint satisfaction problem in which user intent is expressed through hard and soft constraints rather than opaque recommendation models.

Chapter 4 worked through the design, describing a hybrid pipeline in which a large language model converts the user’s description into a structured JSON request, which is then used to filter an SQLite track database and select a final playlist through the Clingo solver. Hard and soft constraints cover artist repetition, banned or required items, and feature distance penalties, with post-processing ordering the selected tracks using a greedy nearest-neighbour algorithm.

The design was then realised in the Implementation chapter (Chapter 5), which described how the pipeline was built in Python, Flask and React. A structured intermediate request approach was adopted after direct LLM-to-ASP generation proved unreliable. Functional testing confirmed that hard constraints were strictly enforced, fallback mechanisms worked correctly, and greedy ordering produced smoother track transitions than a random shuffle.

The results from the comparison study in Chapter 6 showed that IntelliPlaylist outperformed the random baseline in precision and subjective quality, while maintaining comparable novelty. The evaluation also revealed some real limitations in the current implementation. The most notable of these was a dataset gap which caused the `nostalgic 1980s` request to return an empty candidate pool, and a duplicate track defect arising from the same track appearing under multiple album entries in the dataset.

To conclude, the objectives from the beginning of the project have been achieved:

1. To define a constraint structure that allows for playlists to be encoded as a concise set of requirements.
2. To explore methods of introducing flexibility or uncertainty into the playlist generation process.
3. To utilise constraint-solving methods to efficiently and accurately construct playlists, with the ability to dynamically update selections while maintaining stability.

4. To design an algorithm that translates natural-language playlist descriptions into the formal constraint language described in (1).

Appendix A

Evaluation Data

A.1 Per-playlist Ratings

display_id	playlist_id	request	condition	track_count	tracks	precision_count	novelty_count	quality_likert
1	19	upbeat road trip mix	random	10	1. Messias — Dead Fish 2. Welcome Back — Mase 3. Seja Forte — Isadora Pompeo 4. 水戸黄門 — Mr.Children 5. First Class — Jack Harlow 6. Itzy Bitzy Spider (Itzy, Bitzy, Arafia) — Babyboomboom 7. Together — Joseph Tilley	6	5	2
2	13	dark moody late-night playlist	random	10	1. Carrier Signal — Kai Whiston;EDEN 2. Prime Street — Smartface 3. Please Make Up Your Mind — Hank Williams 4. Panis Angelicus — Josh Groban 5. I Miss You (feat. Julia Michaels) — Clean Bandit;Julia Michaels 6. Soy Aquel Viajero — Carlos Di Sarli;Alberto Podestà 7. Zé do Caroco (Ao Vivo) — Seu Jorge	4	7	2
3	15	happy energetic playlist	intelliplylist	10	1. Moosedrilla (feat. DIVINE) — Sidhu Moose Wala;DIVINE 2. チェリーブラッサム — Seiko Matsuda 3. When You're Gone — Shawn Mendes 4. 女唄んでブギ — サザンオールスターズ 5. Rimas Perfectas — Los Caligaris 6. Nüchtern — Anstadslos & Durchgeknallt;Emi Flemming 7. Fee Ra Huri — OMNIA	7	7	4
4	20	chill study music, 15 songs	random	15	1. Left & Right — SEVENTEEN 2. Na Ja — Pav Dharia 3. Poles Apart — Pink Floyd 4. Gravity — Anthony Lazaro 5. December — Neck Deep 6. Krautrock — Surgeon 7. Breathe — Lonely in the Rain	10	9	3
5	4	high-energy gym playlist, no live versions	random	10	1. Reminiscence — Satoshi Takebe 2. A Veces Me Pregunto — DLG 3. Way of the World — Kyle Kinane 4. Bowser — XXXTENTACION;Ski Mask The Slump God 5. Socks & Sandals (Pajama Pants) — Psychostick 6. Electric Garden — Juan Atkins;Moritz Von Oswald 7. Tire Loma Da Nigbehin — Monomono	7	6	3

6	2	upbeat road trip mix	intelliplylist	10	1. Sad Girls (feat. Remy) [Clean Bandit VIP Remix] — Clean Bandit;French The Kid;Remy 2. Zingaat — Ajay-Atul 3. Moon Over Memphis — Balsam Range 4. A Different City — Pickin' On Series;Iron Horse 5. Freedom of Choice - 2009 Remaster — DEVO 6. Move on Up — The Dynamics	9	7	4
7	9	romantic dinner playlist	intelliplylist	10	1. Smoothie Song — Nickel Creek 2. Tuyo (Narcos Theme) - A Netflix Original Series Soundtrack — Rodrigo Amarante 3. Bath Time — Orbital 4. Speedboat - Remastered — Lloyd Cole and the Commotions 5. Parrillera — Trio Hugo Diaz 6. (Ghost) Riders In The Sky — The Ventures	6	7	3
8	17	angry rock songs, 20 tracks	random	20	1. Kimi ga Kureta Mono — Shizuka Kudo 2. Cradledown — Psyche/BFC 3. The Listening Wind — Michael Hoppe;Martin Tillman;Tim Wheatler 4. Shogeki — Yoko Ando 5. Better Than This - Director's Cut Classic Soul Mix — Paul Johnson;Frankie Knuckles;Eric Kupper;Director's Cut 6. Marry Christmas — salem ilese	6	16	2
9	18	chill study music, 15 songs	intelliplylist	15	1. All My Days — Alexi Murdoch 2. Dokhtare Boyerahmadi (Folk Song) — Martin Shamoopour 3. kahazani — When Chai Met Toast 4. 翻る人た ち — Hanare Gumi 5. Wonderful Tonight — Eric Clapton 6. Wonderful Tonight — Eric Clapton 7. Tango-Chaos — Germaine Franco	11	8	4
10	5	sad acoustic songs	intelliplylist	10	1. Blue Christmas — Brenda Lee 2. Go — Prateek Kuhad 3. Nee Vupuri Nee Sothama — Ramakrishnan Murthy 4. Aji kal ve X Awaaz — Jalgo Gill 5. Blue Velvet — Bobby Vinton 6. It Is Well with My Soul — Audrey Assad 7. bouquet — Ichiko Aoba	6	7	3
11	12	angry rock songs, 20 tracks	intelliplylist	20	1. Symphony Of Destruction — Megadeth 2. I Love My Country — Florida Georgia Line 3. All The Small Things — blink-182 4. All The Small Things — blink-182 5. Local God — Panic! At The Disco 6. The Rumbling — SIM 7. Fish Rock — Thaikudam Bridge	9	7	2
12	14	romantic dinner playlist	random	10	1. Ha nocnogwwo nmprey — Muxaww Baww 2. Take It Easy - Sonny Fodera & Mat.Joe Remix (Mixed) — Gershon Jackson;Mat.Joe;Sonny Fodera 3. Sonate IV Opus 180 Pour Violoncelle Et Piano - 3ème Mouvement — Allreza Mashayekhi;Marie-Thérèse Grisenti;Marc Vitantonio;Duo Grisenti Vitantonio 4. Nice & Easy (feat. Rodney P) — Bassnectar;Rodney P	3	6	1
13	10	nostalgic 80s pop	random	10	1. Girls Just Wanna Have Some — Chromatics 2. Inner Acid — Mr. Fingers 3. イヤフォン・ライオット — Shiritsu Ebisu Chugaku 4. Finding — Kiawasch Sahebnassagh 5. Pump Action — Overseer 6. Hackearam-Me (Pagode) - Acústico — Sem Reznha 7. Irretrievable — Dear Seattle	2	8	2
14	6	high-energy gym playlist, no live versions	intelliplylist	10	1. ABC's Song — Melodees 2. Blue Coupe — Twin Peaks 3. Back Where I Belong (feat. Avicii) — Otto Knows;Avicii 4. There Is No Love Between Us Anymore — Pop Will Eat Itself 5. Little Drummer Boy — Four Tops 6. What You Gonna Ask For - Dego — Theo Parrish;Lori;Silentjay;Simon Mavin;Perrin Moss;Paul Bender;Dego	6	7	3
15	3	calm focus music, mostly instrumental	intelliplylist	10	1. All My Days — Alexi Murdoch 2. Joint — Type.Ravi;Loot Fattig 3. Mi Serenata — El Cachivache Quinteto 4. O Re Piya - Flute Version (Instrumental) — Salim - Sulaiman 5. Desde el Alma — Carlos Lombardi;Los Románticos De La Noche 6. Toulumne — Eddie Vedder 7. Carnaval Arequipeno — Raúl García Zárate	9	7	4
16	7	sad acoustic songs	random	10	1. The Power of Love — Gabrielle Aplin 2. Quiero Verte — NAFTA 3. It Came Upon a Midnight Clear — Josh Groban 4. Só Depois de Muito Amor Eu Vou Embora - Ao Vivo — Geraldo Azevedo 5. Residentes Olvidados — Leprosy 6. The Isle Of Innisfree — Celtic Thunder;Emmet Cahill	5	7	2
17	16	happy energetic playlist	random	10	1. Jetlag — Mike Williams;Brooks 2. Mega Funk De Copão na Mão — DJ Ramonstro;SÓ MEGA FUNK;Mc Lelêto 3. Fear No More — The Afters 4. ぐるぐるワンダーランド — SILENT SIREN 5. Bela Gellir Bulur Bizl — Sayedar;Önder Şahin 6. Provação — Felipe Vieira	8	6	4
18	11	nostalgic 80s pop	intelliplylist	10	1. Teri Hogalyan — Vishal Mishra 2. Jeena Jeena — Atif Aslam 3. Inthandham (From "Sita Ramam (Telugu)") — Vishal Chandrashekar;S.P. Charan 4. Pinju Pinju Mazhai — Yuvan Shankar Raja;Sid Sriram 5. Pani Da Rang - Male Vocals — Ayushmann Khurrana 6. O Saathi — Atif Aslam;Arko	0	7	1
19	8	calm focus music, mostly instrumental	random	10	1. Nunca Mas Podre Olvidarte — Los Cuatro Soles 2. The Back Benchers Returns — Thakara;V3K 3. Mary Melody — Mad Caddies 4. Qué Más Pues? — J Balvin;Maria Becerra 5. Entre A Cruz E A Espada — Soweto 6. Cloud Study — Norris Melcher 7. Jack Your Body - 1986 Club Mix — Steve "Silk" Hurley	1	6	1
20	1	dark moody late-night playlist	intelliplylist	10	1. I Know - Original — Hybrid 2. Unstoppable Now — The Phantoms 3. Possession — Portrayal of Guilt 4. Thoughts Like Hammers — Enslaved 5. My LSD Song — Bliss 6. Anima Mea — Sajanka;WHITENO1SE 7. Words — Feint;Laura Brehm	7	7	3

A.2 Defects Observed During Evaluation

A.2.1 Duplicate Track Examples

display_id	playlist_id	request	condition	track_count	tracks
9	18	chill study music, 15 songs	intelliplaylist	15	1. All My Days — Alexi Murdoch 2. Dokhtare Boyerahmadi (Folk Song) — Martin Shamoopour 3. Kahaani — When Chai Met Toast 4. 踊る人たち — Hanare Gumi 5. Wonderful Tonight — Eric Clapton 6. Wonderful Tonight — Eric Clapton 7. Tango-Chaos — Germaine Franco 8. ¿Cómo Fue? — Beny Moré 9. Santa Claus Is Coming To Town — Bill Evans 10. Magnolia — Eric Clapton;John Mayer 11. Had To Fall In Love — The Moody Blues 12. A Feast of Friends — Jim Morrison 13. Tennessee Waltz — Floyd Cramer 14. street beats — Synkop 15. La Cumparsita — Aníbal Troilo;Roberto Grela

display_id	playlist_id	request	condition	track_count	tracks
11	12	angry rock songs, 20 tracks	intelliplaylist	20	1. Symphony Of Destruction — Megadeth 2. I Love My Country — Florida Georgia Line 3. All The Small Things — blink-182 4. All The Small Things — blink-182 5. Local God — Panic! At The Disco 6. The Rumbling — SiM 7. Fish Rock — Thaikkudam Bridge 8. Blood // Water — grandson 9. Street Fighting Man — Rod Stewart 10. Christmas All Over Again — The Goo Goo Dolls 11. Mary On A Cross — Ghost 12. Shut Up and Dance — WALK THE MOON 13. Clarity — Vance Joy 14. Khudi — The Local Train 15. Angel - Metro Mix Radio Edit — Lionel Richie 16. Sweater Weather — The Neighbourhood 17. human - Passion Pit Remix — Christina Perri;Passion Pit 18. Electric Love — BØRNS 19. Come As You Are — Nirvana 20. unravel — TK from Ling tosite signature

A.2.2 Empty Candidate Pool

```
=====
Pass 1: IntelliPlaylist generation
=====
OK [10 tracks] "happy energetic playlist"
OK [15 tracks] "chill study music, 15 songs"
OK [10 tracks] "sad acoustic songs"
OK [10 tracks] "high-energy gym playlist, no live versions"
OK [ 0 tracks] "nostalgic 80s pop"
OK [10 tracks] "calm focus music, mostly instrumental"
OK [20 tracks] "angry rock songs, 20 tracks"
OK [10 tracks] "romantic dinner playlist"
OK [10 tracks] "upbeat road trip mix"
OK [10 tracks] "dark moody late-night playlist"
```

References

- [1] BAHADURE, N. B., VISHWAKARMA, M., KURHEKAR, M. P., PATNI, J. C., AND PATIL, P. D. Machine learning-based music classification and recommendation system from spotify. *International Journal of Computer Information Systems and Industrial Management Applications* 17 (2025), 12.
- [2] BOGDANOV, D., HARO, M., FUHRMANN, F., XAMBÓ, A., GÓMEZ, E., AND HERRERA, P. Semantic audio content-based music recommendation and visualization based on user preference examples. *Information Processing & Management* 49, 1 (2013), 13–33.
- [3] CASEY, M. A., VELTKAMP, R., GOTO, M., LEMAN, M., RHODES, C., AND SLANEY, M. Content-based music information retrieval: Current directions and future challenges. *Proceedings of the IEEE* 96, 4 (2008), 668–696.
- [4] CHEN, Y. A music recommendation system based on collaborative filtering and SVD. In *2022 IEEE Conference on Telecommunications, Optics and Computer Science (TOCS)* (2022), IEEE, pp. 1510–1513.
- [5] CIOCCA, S. Spotify’s discover weekly: How machine learning finds your new music. <https://hackernoon.com>, 2017. HackerNoon article.
- [6] FAYYAZ, Z., EBRAHIMIAN, M., NAWARA, D., IBRAHIM, A., AND KASHEF, R. Recommendation systems: Algorithms, challenges, metrics, and business opportunities. *Applied Sciences* 10, 21 (2020), 7748.
- [7] FELFERNIG, A., AND BURKE, R. Constraint-based recommender systems. In *Proceedings of the 10th International Conference on Electronic Commerce* (New York, NY, USA, 2008), ACM, pp. 1–10.
- [8] FIGUEROA, R. Spotify 1.2m+ songs. Kaggle dataset, 2020.
- [9] GEBSER, M., KAMINSKI, R., AND KAUFMANN, B. Multi-criteria optimization in answer set programming. In *Technical Communications of the Twenty-Seventh International Conference on Logic Programming (ICLP’11)* (2011), pp. 1–10.
- [10] GOOGLE. Gemini api. <https://ai.google.dev/gemini-api/docs>, 2025. Accessed 2025.
- [11] HU, Y., KOREN, Y., AND VOLINSKY, C. Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE International Conference on Data Mining* (2008), IEEE.
- [12] HUGGING FACE. Hugging face. <https://huggingface.co/>, 2025. Accessed 2025.
- [13] LIFSCHITZ, V. *Answer Set Programming*. Springer International Publishing, Cham, 2019.

- [14] LÜ, L., MEDO, M., YEUNG, C. H., ZHANG, Y., ZHANG, Z., AND ZHOU, T. Recommender systems. *Physics Reports* 519, 1 (2012), 1–49.
- [15] MAGADUM, H., AZAD, H. K., PATEL, H., AND R., R. H. Music recommendation using dynamic feedback and content-based filtering. *Multimedia Tools and Applications* 83, 32 (2024), 77469–77488.
- [16] MILLECAMP, M., HTUN, N. N., JIN, Y., AND VERBERT, K. Controlling Spotify recommendations. In *Proceedings of the 26th Conference on User Modeling, Adaptation and Personalization* (New York, NY, USA, 2018), ACM, pp. 101–109.
- [17] MURCIEGO, L., JIMÉNEZ-BRAVO, D., ROMÁN, V., SANTANA, J., AND GARCÍA, M. Context-aware recommender systems in the music domain: A systematic literature review. *Electronics* 10 (2021), 1555.
- [18] NIRJON, S., DICKERSON, R. F., LI, Q., ASARE, P., STANKOVIC, J. A., HONG, D., ZHANG, B., JIANG, X., SHEN, G., AND ZHAO, F. Musicalheart. In *Proceedings of the 10th ACM Conference on Embedded Network Sensor Systems* (New York, NY, USA, 2012), ACM, pp. 43–56.
- [19] NORTH, A. C., HARGREAVES, D. J., AND HARGREAVES, J. J. Uses of music in everyday life. *Music Perception* 22, 1 (2004), 41–77.
- [20] PANDYA, M. Spotify tracks dataset. Kaggle dataset, <https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>, 2022. Accessed 2025.
- [21] PICHL, M., ZANGERLE, E., AND SPECHT, G. Towards a context-aware music recommendation approach: What is hidden in the playlist name? In *2015 IEEE International Conference on Data Mining Workshop (ICDMW)* (2015), IEEE, pp. 1360–1365.
- [22] RICCI, F., ROKACH, L., AND SHAPIRA, B. *Recommender Systems Handbook*. Springer, 2015.
- [23] RIDLEY, M. Using folk theories of recommender systems to inform human-centered explainable AI (HCXAI). *The Canadian Journal of Information and Library Science* 46, 2 (2023), 1–19.
- [24] RUSSELL, J. A. A circumplex model of affect. *Journal of Personality and Social Psychology* 39, 6 (1980), 1161–1178.
- [25] SPOTIFY. Spotify — about spotify. <https://www.spotify.com>, 2018. Accessed 2025.
- [26] SPOTIFY. Web api. <https://developer.spotify.com>, 2025. Accessed 2025.
- [27] THAYER, R. E. Toward a psychological theory of multidimensional activation (arousal). *Motivation and Emotion* 2, 1 (1978), 1–34.
- [28] TZANETAKIS, G., AND COOK, P. Musical genre classification of audio signals. *IEEE Transactions on Speech and Audio Processing* 10, 5 (2002), 293–302.
- [29] WIAFE, A., SIERANOJA, S., BHUIYAN, A., AND FRÄNTI, P. Emotional response to music: the Emotify+ dataset. *EURASIP Journal on Audio, Speech, and Music Processing* 2025, 1 (2025).
- [30] YANG, Y., AND CHEN, H. H. Machine recognition of music emotion: A review. *ACM Transactions on Intelligent Systems and Technology* 3, 3 (2012).